

LINGUAGEM DE PROGRAMAÇÃO II

Apostila completa da disciplina — teoria por semana (4 etapas), exercícios práticos, testes rápidos e avaliações (A1 · A2 · A3) · VB6 + MySQL

72 aulas · 24 semanas

3 aulas semanais (50 min)

60 h presenciais

10 h não presenciais (AVA)

12 módulos · 6 partes

3 avaliações · 100 pts

VERSÃO DO PROFESSOR · CONTÉM GABARITOS

Material elaborado a partir do Plano de Ensino e do Cronograma oficiais da disciplina · Setembro / 2026 · Docente responsável: _____

Apresentação da apostila

Esta apostila foi elaborada a partir do **Plano de Ensino e do Cronograma oficiais** da disciplina **Linguagem de Programação II** (Curso Técnico em Informática, Turma 2/2026): **96 aulas em 24 semanas, 4 aulas semanais**. Ela reúne a teoria de cada semana nas **4 etapas oficiais** (conceitos > demonstração > prática > fixação), exercícios de laboratório, testes rápidos e os modelos das avaliações A1, A2 e A3.

O ambiente de trabalho do curso (escolha documentada — S) é **Visual Basic 6.0** com o SGBD **MySQL** acessado por **ODBC/ADO**; o plano B para laboratórios sem servidor é o Access, com os mesmos conceitos. Todos os códigos de exemplo seguem esse ambiente.

- **Semanas 1–3:** ambiente, SGBD (instalação/configuração) e conexão;
- **Semanas 4–6:** linguagem SQL (fundamentos, manipulação e consultas);
- **Semanas 7–10:** programação estruturada, vetores, matrizes e registros;
- **Semanas 11–14:** Strings, modularização e arquivos/textos;
- **Semanas 15–18:** dados Cliente/Servidor (ADO), backup e recuperação;
- **Semanas 19–24:** relatórios, documentos fiscais, disco de instalação, gráficos e projeto integrador.

Como estudar com esta apostila (aula invertida)

Antes da semana: leia a teoria e tente o primeiro exercício; em sala: laboratório em duplas e correção dialogada; depois: teste rápido da semana e atividades do AVA. Programação se aprende digitando: todo exemplo desta apostila deve ser executado, quebrado e consertado por você.

DICA

Itens marcados com **(S)** são **propostas pedagógicas** de organização (datas de avaliações, entregas, escolha de ambiente) — não especificadas no Plano de Ensino original. Demais informações reproduzem o Plano de Ensino e o Cronograma oficiais.

Ícones e caixas usadas neste material

Caixa	O que significa	Quando usar
Conceito-chave	Definição importante que provavelmente cairá nas avaliações.	Leia e memorize; anote no caderno.
Analogia	Comparação com situações do cotidiano.	Use para explicar com suas palavras.
Dica	Sugestão prática ou boa prática profissional.	Aplique nos laboratórios.
Atenção	Erro comum ou ponto de risco (dados, laços, arquivos).	Leia antes de executar.
Código	Exemplo VB6/SQL/shell comentado.	Digite e execute — nunca apenas leia!
Exercícios	Atividades práticas e escritas do módulo.	Resolva em sala e complete no AVA.
Teste rápido	5 questões objetivas de verificação da semana/módulo.	Responda sem consultar; depois corrija.

DISCIPLINA · CARGA HORÁRIA · MÓDULOS · AVALIAÇÕES**Visão geral do curso**

Item	Informação
Disciplina	Linguagem de Programação II — Curso Técnico em Informática
Turma	Turma 2 / 2026 · Eixo Tecnológico: Informação e Comunicação
Carga horária presencial	80 h presenciais (aulas teóricas e práticas)
Carga horária não presencial	14 h não presenciais (atividades orientadas no AVA)
Estrutura	96 aulas · 24 semanas · 4 aulas semanais (períodos de 50 min)

EMENTA (SÍNTESE OFICIAL)

Instalação e configuração dos requisitos de SGBD para a Linguagem; Conexão com Banco de Dados; Programação de instruções SQL; Programação gráfica em ambiente DOS; Criação de programas usando linguagem de programação estruturada; Arquivos e textos como forma de transferência de dados; Manipulação de Dados Cliente/Servidor; Backup e Recuperação de Banco de Dados; Geração de Relatórios; Impressão e Geração de Documentos Fiscais; Criação do Disco de Instalação do Programa; Estruturação de dados com vetor, Matriz e registro; Técnicas de modularização de programas; Variáveis do tipo String; Recursos gráficos.

Módulos do semestre (distribuição oficial dos conteúdos)

#	Módulo	Conteúdo principal	Aulas	Semanas
1	Ambientação e linguagem estruturada	Apresentação da disciplina; programação estruturada; arquitetura do projeto; organização do ambiente	1–4	1
2	SGBD: instalação e conexão	Requisitos do SGBD; instalação; configuração; conexão; parâmetros; tratamento de falhas	5–12	2–3
3	SQL: fundamentos e manipulação	Instruções SQL; criação/consulta; INSERT, UPDATE, DELETE; integração ao programa	13–20	4–5
4	SQL: consultas e filtros	SELECT com WHERE, ORDER BY, agrupamentos; organização de consultas; cenários de dados	21–24	6
5	Programação estruturada	Sequência, seleção (If/Select Case) e repetição (For/Do/While); organização lógica	25–28	7
6	Vetores e matrizes	Declaração, preenchimento, leitura e processamento de arrays uni e bidimensionais	29–36	8–9
7	Registros	Type/End Type; agrupamento de informações relacionadas; uso no programa	37–40	10
8	Strings	Variáveis String; armazenamento, leitura e manipulação de textos (Len, Mid, UCase, Trim...)	41–44	11

#	Módulo	Conteúdo principal	Aulas	Semanas
9	Modularização	Sub, Function, parâmetros, retorno; Modules; reutilização e organização	45–48	12
10	Arquivos e textos	Leitura/gravação de arquivos; For Output/Input/Append; integração com dados do programa	49–56	13–14
11	Manipulação de dados Cliente/Servidor	ADO/ODBC; Recordset; fluxo aplicação × banco; validação e tratamento de problemas	57–64	15–16
12	Backup e recuperação	Procedimentos de cópia (mysqldump); restauração; validação após recuperação	65–72	17–18
13	Relatórios e documentos fiscais	Geração de relatórios; seleção e organização de dados; objeto Printer; documentos fiscais	73–80	19–20
14	Disco de instalação e gráficos	Package & Deployment Wizard; recursos gráficos (PictureBox; herança DOS)	81–88	21–22
15	Projeto integrador e consolidação	Integração geral; revisão; apresentação do projeto; avaliação final	89–96	23–24

Metodologia (estratégias de ensino oficiais)

- Exposição de conceitos, exemplos e atividades em sala (aulas dialogadas).
- Aula invertida: estudo prévio em casa (bibliografia indicada) e aplicação em sala.
- Exercícios individuais e em grupo em cada tópico, com aplicação prática.
- Estudos de caso para análise crítica (cenários reais de dados e sistemas).
- Laboratórios práticos ao longo de todos os tópicos.
- Projeto integrador nas semanas 23–24 consolidando os conteúdos.

Recursos didáticos

- Laboratório de informática com Windows e Visual Basic 6.0 instalado.
- Servidor MySQL (ou plano B: Access) nas máquinas do laboratório.
- Recursos audiovisuais (projetor) e quadro branco.
- Portal Acadêmico (AVA): tarefas, fóruns e questionários.
- Google Drive, formulários e e-mail.
- Biblioteca virtual e WhatsApp (comunicação).

Atividades extraclasse orientadas — 10 h (proposta — S)

Fase	Semanas	Horas	Atividades previstas
Semanas 2–4	2 h	Preparação do ambiente e estudo orientado sobre SGBD e conexão.	
Semanas 7–10	3 h	Exercícios de programação estruturada, vetor, matriz e registro.	
Semanas 11–14	3 h	Atividades sobre String, modularização e arquivos/textos.	
Semanas 15–18	2 h	Estudo orientado sobre Client e Servidor, backup e recuperação.	

Fase	Semanas	Horas	Atividades previstas
Semanas 19–22	2 h	Preparação de relatórios, documentos e instalação.	
Semanas 23–24	2 h	Consolidação e preparação do projeto/avaliação final.	

Avaliações — 100 pontos no semestre

Avaliação	Pontos	Período	Composição (proposta — S)
1ª Avaliação	30 pts	Semanas 1 a 8	Lista prática de SQL + estruturas (10 pts — entrega semana 6) + atividades AVA (5 pts) + teste escrito-prático de SGBD/SQL/estruturas (15 pts — semana 8). (S)
2ª Avaliação	30 pts	Semana 16	Prova obrigatória (pode ser em duplas ou com consulta — Plano de Ensino): Strings, modularização, arquivos e Cliente/Servidor.
3ª Avaliação	40 pts	Semana 24	Prova escrita final individual e cumulativa (30 pts) + projeto integrador e apresentação (10 pts — critério do professor).

Sumário

Apresentação da apostila	2
Visão geral do curso	3
PARTE I — Ambiente, SGBD e Conexão	9
Módulo 1 — Ambientação e linguagem de programação estruturada (Aulas 1 a 4)	11
Módulo 2 — SGBD: instalação, configuração e conexão (Aulas 5 a 12)	17
PARTE II — Linguagem SQL	22
Módulo 3 — SQL: fundamentos e manipulação de dados (Aulas 13 a 20)	24
Módulo 4 — SQL — consultas e filtros (Aulas 21 a 24)	29
PARTE III — Programação Estruturada e Estruturas de Dados	33
Módulo 5 — Programação estruturada no VB6 (Aulas 25 a 28)	35
Módulo 6 — Vetores e matrizes (Aulas 29 a 36)	40
Módulo 7 — Registros: agrupando informações relacionadas (Aulas 37 a 40)	46
PARTE IV — Strings, Modularização e Arquivos	50
Módulo 8 — Variáveis do tipo String (Aulas 41 a 44)	52
Módulo 9 — Modularização: Sub, Function e Modules (Aulas 45 a 48)	57
Módulo 10 — Arquivos e textos como transferência de dados (Aulas 49 a 56)	62
PARTE V — Dados Cliente/Servidor e Administração do Banco	67
Módulo 11 — Manipulação de dados Cliente/Servidor (ADO) (Aulas 57 a 64)	69
Módulo 12 — Backup e recuperação de banco de dados (Aulas 65 a 72)	75
PARTE VI — Saídas, Distribuição, Gráficos e Projeto Integrador	79
Módulo 13 — Relatórios, impressão e documentos fiscais (Aulas 73 a 80)	81
Módulo 14 — Disco de instalação e recursos gráficos (Aulas 81 a 88)	86
Módulo 15 — Projeto integrador e consolidação final (Aulas 89 a 96)	91
Projeto Final Integrador	95
Avaliações — 100 pontos no semestre	97

Glossário	109
Bibliografia	111

PARTE I

AMBIENTE, SGBD E CONEXÃO

AULAS 1 A 12 • SEMANAS 1 A 3

Módulo 1 — Ambientação e linguagem de programação estruturada

Módulo 2 — SGBD: instalação, configuração e conexão

As semanas 2–3 instalam e validam o ambiente (SGBD + conexão): é a base de todo o resto do semestre — não pule os laboratórios.

MÓDULO 1 · AULAS 1 A 4 · SEMANA 1

Ambientação e linguagem de programação estruturada

PARTE I — AMBIENTE, SGBD E CONEXÃO

OBJETIVOS DE APRENDIZAGEM

- ✓ Compreender o escopo da disciplina e a arquitetura do projeto final.
- ✓ Reconhecer os três pilares da programação estruturada: sequência, seleção e repetição.
- ✓ Identificar os componentes do ambiente: IDE VB6, SGBD MySQL, driver ODBC e biblioteca ADO.
- ✓ Organizar o ambiente de trabalho e executar o primeiro programa VB6.

AULA 1

Ambientação, linguagem estruturada e primeiro programa VB6

O que vamos construir neste semestre

Em Linguagem de Programação II, você sai do “programa que só calcula” e entra no mundo dos **dados que persistem**: programas que conversam com um **banco de dados**, consultam com **SQL**, guardam informações em **arquivos**, imprimem **relatórios e documentos** e viram um **pacote de instalação** completo. Tudo isso se junta no projeto integrador das semanas 23–24.

Programação estruturada

Estilo de programar baseado em três estruturas de controle: **sequência** (passo a passo), **seleção** (If/Select Case — decidir) e **repetição** (For/Do While — repetir). Sem “pulos” desorganizados (o famoso GOTO): o código fica legível, testável e fácil de manter.

A linguagem e o banco escolhidos (S)

Trabalharemos com **Visual Basic 6.0 (VB6)** como linguagem estruturada — com vetores, matrizes, registros (Type), Strings, Sub/Function, arquivos e gráficos — e o SGBD **MySQL**, acessado por **ODBC/ADO**. Se o laboratório não tiver servidor, usamos o plano B documentado: Access (.mdb), mesmos conceitos.

Componentes do ambiente

Componente	Papel no curso
IDE VB6	Editor + compilador + construtor de telas (formulários)
MySQL Server	SGBD: guarda bancos, tabelas e executa SQL
Driver MySQL ODBC	Ponte que deixa o VB6 conversar com o MySQL
ADO (ActiveX Data Objects)	Biblioteca VB6 que envia SQL e recebe resultados (Recordset)

Arquitetura do projeto final

- **Camada de dados:** banco MySQL com tabelas do tema escolhido;
- **Camada de acesso:** módulo de conexão + comandos SQL (ADO);
- **Camada de lógica:** Modules com Sub/Function (modularização);
- **Camada de apresentação:** formulários VB6 + relatórios/Printer;
- **Distribuição:** pacote de instalação (Package & Deployment Wizard).

Contrato da disciplina

- 4 aulas por semana em 4 etapas: conceitos > demonstração > prática > fixação;
- Avaliações: A1 (30) semanas 1–8 · A2 (30) semana 16 · A3 (40) semana 24;
- 14 h não presenciais no AVA, distribuídas conforme o cronograma;
- Diagnóstico inicial nesta semana: vale ponto de participação, não de nota.

Anatomia de um programa VB6

No VB6, o código vive em **módulos** (Modules) e **formulários** (Forms). Subs executam ações; Functions retornam valores. O ponto de partida clássico é o evento **Form_Load** (quando a janela abre) ou um botão clicado.

Form1 – primeiro programa (sequência + saída)

VB

```
Private Sub Form_Load()  
    ' sequência: passo a passo, de cima para baixo  
    Dim nome As String  
    nome = "Turma 2/2026"  
    Print "Olá, " & nome & "!" ' saída no formulário  
    Print "Disciplina: Linguagem de Programação II"  
End Sub
```

As três estruturas, em um relance

Seleção e repetição (prévia das semanas 7+)

VB

```
' SELEÇÃO: decidir  
If nota >= 7 Then  
    Print "Aprovado"  
Else  
    Print "Recuperação"  
End If  
  
' REPETIÇÃO: repetir  
Dim i As Integer  
For i = 1 To 5  
    Print i  
Next i
```

Organização do ambiente de trabalho

1. Crie a pasta do semestre: **C:\LP2** (ou Documents\LP2);
2. Dentro dela, uma pasta por semana: **S01, S02...** e, no fim, **PROJETO**;

3. Salve todo projeto VB6 (.vbp) dentro da pasta da semana correspondente;
4. Nomes sem acento/espço: **Conexao.bas**, **frmAlunos.frm**;
5. Faça backup semanal da pasta no Drive (hábito de profissional).

ATENÇÃO

VB6 é antigo e sensível a caminhos com acentos/espços e a resoluções muito altas. Padronize: pastas simples, arquivos .vbp versionados por data e sempre FECHAR a IDE pelo menu (evita projetos corrompidos).

EXERCÍCIOS — MÓDULO 1

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Diagnóstico de lógica**ESCRITO**

Sem computador: escreva em pseudocódigo (ou desenhe) um algoritmo que leia 3 notas, calcule a média e imprima a situação (≥ 7 aprovado; ≥ 5 recuperação; senão reprovado). Identifique no seu algoritmo onde está cada estrutura: sequência, seleção e repetição (use repetição para ler as 3 notas).

RESULTADO ESPERADO

Algoritmo com laço para leitura das 3 notas, If/Elseif encadeado para a situação e identificação correta das três estruturas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção dialogada no projetor com 2–3 soluções de alunos; aproveite para nivelar a turma.

EXERCÍCIO 2 — Organização do ambiente**PRÁTICO · LABORATÓRIO**

Crie a estrutura de pastas do semestre (LP2\S01..S04 + PROJETO), abra o VB6, crie um projeto novo salvo em LP2\S01\ola.vbp e faça o Form_Load imprimir seu nome e a data de hoje (use Date).

RESULTADO ESPERADO

Árvore de pastas criada; projeto .vbp salvo no caminho correto; formulário imprimindo nome e data ao rodar (F5).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir caminho do .vbp (muitos salvam na área de trabalho) e uso de & para concatenar.

EXERCÍCIO 3 — As três estruturas na prática**PRÁTICO · LABORATÓRIO**

No mesmo projeto: adicione um botão que leia um número por InputBox e use If para dizer se é par ou ímpar (Mod); adicione outro botão que imprima com For a tabuada desse número de 1 a 10.

RESULTADO ESPERADO

Dois botões funcionais: par/impar com Mod e tabuada com For...Next usando & na montagem das linhas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Ponto de atenção: Val(InputBox(...)) para converter texto em número; circular e ajudar com a sintaxe do For.

TESTE RÁPIDO — MÓDULO 1: Ambientação e linguagem de programação estruturada

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Programação estruturada se apoia em três estruturas:

- (A) entrada, processamento e saída
- (B) sequência, seleção e repetição
- (C) variáveis, constantes e funções
- (D) telas, relatórios e bancos

RESPOSTA: B

Sequência executa em ordem; seleção decide; repetição repete blocos.

2. No ambiente do curso (S), quem conversa diretamente com o MySQL a partir do VB6 é:

- (A) o objeto Printer
- (B) o driver ODBC + biblioteca ADO
- (C) o Package & Deployment Wizard
- (D) o explorador de arquivos

RESPOSTA: B

ODBC faz a ponte VB6-MySQL; ADO envia SQL e recebe Recordset.

3. Um Sub difere de uma Function porque:

- (A) Sub retorna valor
- (B) Function retorna valor
- (C) Sub só roda em Modules
- (D) não há diferença

RESPOSTA: B

Function devolve resultado ao chamador; Sub apenas executa ações.

4. O padrão de organização de pastas adotado no curso é:

- (A) tudo na área de trabalho
- (B) LP2\S01..S24 + PROJETO, sem acentos/espacos
- (C) uma pasta por aluno no pendrive
- (D) pastas com nome das disciplinas por extenso

RESPOSTA: B

Caminhos simples evitam bugs no VB6 e facilitam backup/entrega.

5. Para converter o texto do InputBox em número usamos:

- (A) Str()
- (B) Val()
- (C) Mid()
- (D) Len()

RESPOSTA: B

Val converte string em número; Str faz o caminho inverso.

MÓDULO 2 · AULAS 5 A 12 · SEMANAS 2 A 3

SGBD: instalação, configuração e conexão

PARTE I — AMBIENTE, SGBD E CONEXÃO

OBJETIVOS DE APRENDIZAGEM

- ✓ Explicar o que é um SGBD e instalar/configurar o MySQL no laboratório.
- ✓ Validar o ambiente com comandos básicos do cliente MySQL.
- ✓ Compreender conexão: parâmetros, abertura, fechamento e string de conexão.
- ✓ Tratar falhas básicas de conexão no VB6 (On Error).

AULA 2**SGBD — instalação e configuração (semana 2)****O que é um SGBD****SGBD — Sistema Gerenciador de Banco de Dados**

Software que **armazena, organiza, protege e controla o acesso** aos dados: cria tabelas, executa consultas, gerencia usuários e garante integridade. Exemplos: MySQL, PostgreSQL, SQL Server, Access. No curso: **MySQL** (gratuito).

Sem SGBD, cada programa inventa seu próprio jeito de guardar dados (arquivos soltos, planilhas...). Com SGBD, todos os programas falam a mesma língua: **SQL**.

Instalação passo a passo (laboratório)

1. Baixe o instalador do MySQL (mysql-installer-community) do site oficial ou da pasta da rede;
2. Execute e escolha o tipo **Server only** (ou Developer padrão se houver espaço);
3. Configuração: authentication padrão, **senha do root** (anote! ex.: aluno123), serviço Windows MySQL habilitado (inicia com o sistema);
4. Conclua e abra o **MySQL Command Line Client** (ou `mysql -u root -p` no prompt);
5. Valide: **SHOW DATABASES;** deve listar information_schema, mysql, performance_schema, sys.

Primeiros comandos no cliente MySQL

SQL

```
mysql -u root -p -- entra no servidor (digite a senha)
SHOW DATABASES; -- lista os bancos existentes
CREATE DATABASE escola; -- cria nosso banco de estudo
USE escola; -- seleciona o banco ativo
SELECT VERSION(); -- confirma a versão instalada
```

DICA

Toda instrução SQL termina com **ponto e vírgula**. E todo comando de administração (SHOW, CREATE DATABASE, USE) é um ótimo exercício de digitação: erro de uma letra = mensagem de erro educativa.

ATENÇÃO

Senha do root perdida = laboratório parado. Padrão da turma + senha anotada no caderno e no caderno do professor. Se o serviço não subir: Serviços do Windows > MySQL80 > Iniciar.

AULA 3**SGBD — conexão: parâmetros, abertura e falhas (semana 3)****O que é uma conexão**

Conectar é **abrir um canal** entre o programa (cliente) e o SGBD (servidor). Para isso, o cliente informa: **driver** (quem traduz), **servidor** (onde), **banco** (qual), **usuário e senha** (credenciais). No VB6, isso vira uma **connection string** passada ao objeto **ADODB.Connection**.

Modulo Conexao.bas — abrir e fechar com tratamento

VB

```
Public cn As ADODB.Connection

Public Sub AbrirConexao()
On Error GoTo Falha
Set cn = New ADODB.Connection
cn.ConnectionString = "Driver={MySQL ODBC 8.0 Driver};" & _
"Server=localhost;Database=escola;" & _
"Uid=root;Pwd=aluno123;"
cn.Open
Print "Conectado ao MySQL!"
Exit Sub
Falha:
MsgBox "Falha de conexão: " & Err.Description, vbCritical
End Sub

Public Sub FecharConexao()
If Not cn Is Nothing Then cn.Close
End Sub
```

Parâmetros da string de conexão

Parâmetro	Significado	Exemplo
Driver	Driver ODBC instalado	{MySQL ODBC 8.0 Driver}
Server	Endereço do servidor	localhost (ou IP da rede)
Database	Banco selecionado ao conectar	escola
Uid / Pwd	Usuário e senha	root / aluno123

Tratamento básico de falhas

- **On Error GoTo Falha:** desvia para o rótulo se algo der errado;
- **Err.Description:** mensagem técnica do erro (mostre ao usuário de forma amigável);
- Falhas típicas: serviço parado, senha errada, driver ausente, banco inexistente;

- Sempre **fechar** a conexão ao terminar (FecharConexao) — conexão aberta é recurso vazando.

Conexão = ligação telefônica

Discar (cn.Open) exige número certo (Server), ramal (Database) e identificação (Uid/Pwd). Se a linha está muda (serviço parado), você desliga e avisa (On Error). E ao terminar a conversa, desliga (cn.Close) — ninguém deixa o telefone fora do gancho a noite toda.

EXERCÍCIOS — MÓDULO 2

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Instalação validada

PRÁTICO · LABORATÓRIO

Instale (ou valide) o MySQL na sua máquina: entre no cliente, rode `SHOW DATABASES;`, crie o banco **escola** e dentro dele a tabela **turma(id INT PRIMARY KEY AUTO_INCREMENT, nome VARCHAR(40))**. Cole a saída no caderno/Drive.

RESULTADO ESPERADO

Banco escola e tabela turma criados sem erros; saída do `SHOW TABLES;` conferindo.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Checklist visual por máquina: cliente abre, banco existe, tabela existe. Anote as máquinas com problema para a TI (tutorial de laboratório).

EXERCÍCIO 2 — Conexão VB6 > MySQL

PRÁTICO · LABORATÓRIO

Crie o projeto ConexaoS03 com o módulo Conexao.bas (modelo da aula) e um formulário com botões Conectar/Desconectar que mostrem o status (conectado/falhado) em um Label.

RESULTADO ESPERADO

Botão Conectar abre cn e mostra 'Conectado'; Desconectar fecha; com o serviço parado, MsgBox amigável com Err.Description.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Teste os 4 cenários de falha em duplas (serviço, senha, driver, banco). Cada dupla registra qual erro provocou e qual mensagem apareceu.

EXERCÍCIO 3 — Mapa da conexão

ESCRITO

Desenhe o fluxo cliente/servidor da conexão (VB6 > ODBC > MySQL) e explique com suas palavras o papel de cada parâmetro da connection string e o que acontece em cada falha simulada no laboratório.

RESULTADO ESPERADO

Diagrama com as 3 camadas e legenda correta dos parâmetros; explicação coerente das falhas (serviço/senha/driver/banco).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Use os melhores diagramas para montar o pôster da sala (fixação visual do semestre).

TESTE RÁPIDO — MÓDULO 2: SGBD: instalação, configuração e conexão

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Um SGBD existe para:

- (A) substituir o sistema operacional
- (B) armazenar, organizar e controlar o acesso aos dados
- (C) compilar programas VB6
- (D) desenhar formulários

RESPOSTA: B

É o gerente dos dados: criação, consulta, segurança e integridade.

2. O comando que lista os bancos do servidor é:

- (A) LIST ALL;
- (B) SHOW DATABASES;
- (C) SELECT BANKS;
- (D) DIR /B;

RESPOSTA: B

SHOW DATABASES; no cliente MySQL.

3. Na string de conexão, o parâmetro Database indica:

- (A) o nome do driver
- (B) o banco selecionado ao conectar
- (C) a porta do servidor
- (D) o nome do formulário

RESPOSTA: B

Server = onde; Database = qual banco; Uid/Pwd = quem.

4. On Error GoTo Falha serve para:

- (A) apagar erros do código
- (B) desviar a execução para um rótulo quando ocorrer erro
- (C) impedir erros de sintaxe
- (D) reiniciar o programa

RESPOSTA: B

Tratamento de erro em tempo de execução; Err.Description detalha.

5. Ao terminar de usar o banco, o programa deve:

- (A) deixar a conexão aberta para a próxima vez
- (B) fechar a conexão (cn.Close)
- (C) desinstalar o driver
- (D) reiniciar o MySQL

RESPOSTA: B

Conexão aberta consome recurso do servidor; feche sempre.

PARTE II

LINGUAGEM SQL

AULAS 13 A 24 · SEMANAS 4 A 6

Módulo 3 — SQL: fundamentos e manipulação de dados

Módulo 4 — SQL — consultas e filtros

MÓDULO 3 · AULAS 13 A 20 · SEMANAS 4 A 5**SQL: fundamentos e manipulação de dados**

PARTE II — LINGUAGEM SQL

OBJETIVOS DE APRENDIZAGEM

- ✓ Diferenciar DDL (estrutura) de DML (dados) e escrever CREATE TABLE adequado.
- ✓ Inserir, alterar e excluir dados com INSERT, UPDATE e DELETE seguros.
- ✓ Consultar dados com SELECT simples.
- ✓ Integrar instruções SQL ao programa VB6 via cn.Execute.

AULA 4**SQL — fundamentos: criar e consultar (semana 4)****Duas famílias de comandos**

DDL x DML

Família	Para que	Comandos
DDL (definição)	criar/alterar a ESTRUTURA	CREATE, ALTER, DROP
DML (manipulação)	cuidar dos DADOS	INSERT, UPDATE, DELETE, SELECT

Estrutura bem definida: tipos e chave primária

SQL

```
USE escola;

CREATE TABLE alunos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(40) NOT NULL,
  curso VARCHAR(20),
  nota DECIMAL(3,1),
  nacido DATE
);
```

Chave primária e AUTO_INCREMENT

A **PRIMARY KEY** identifica cada linha de forma única (nunca repete, nunca nula). Com **AUTO_INCREMENT**, o próprio MySQL numera as linhas: 1, 2, 3... — você não informa o id no INSERT.

Consultar: o SELECT mínimo

SQL

```
SELECT nome, nota FROM alunos; -- colunas escolhidas
SELECT * FROM alunos; -- todas as colunas
```

Boas práticas de digitação SQL

- Comandos em MAIÚSCULAS, nomes de tabela/coluna em minúsculas (legibilidade);

- Uma cláusula por linha a partir do FROM/WHERE;
- Comentários com `--` explicando a intenção;
- Guarde scripts .sql por semana: eles viram seu histórico de estudo.

AULA 5**SQL — manipulação: INSERT, UPDATE, DELETE + VB6 (semana 5)**

Inserir, alterar, excluir

SQL

```
INSERT INTO alunos (nome, curso, nota)
VALUES ('Ana Silva', 'Informática', 8.5);

UPDATE alunos SET nota = 7.0 WHERE id = 2; -- SEMPRE com WHERE!

DELETE FROM alunos WHERE id = 3; -- SEMPRE com WHERE!
```

ATENÇÃO

UPDATE/DELETE sem WHERE afetam TODAS as linhas. Antes de executar, rode o SELECT com o mesmo WHERE para conferir quais linhas serão tocadas. Regra de laboratório: SELECT antes, manipulação depois.

SQL dentro do programa VB6

Com a conexão aberta, o VB6 envia SQL com **cn.Execute**. Comandos de manipulação não retornam tabela; consultas retornam um **Recordset** (veremos a fundo na semana 6 e no módulo 11).

Modulo SqlAlunos.bas – manipulação pelo programa

VB

```
Public Sub InserirAluno(nome As String, nota As Single)
cn.Execute "INSERT INTO alunos (nome, nota) VALUES ('" & _
nome & "', " & nota & ")"
End Sub

Public Sub SubirNota(id As Integer, nova As Single)
cn.Execute "UPDATE alunos SET nota = " & nova & _
" WHERE id = " & id
End Sub
```

DICA

Montar SQL por concatenação (&) é o jeito didático agora; em sistemas reais usamos parâmetros/validação para evitar injeção de SQL — consciência que já nasce aqui.

EXERCÍCIOS — MÓDULO 3

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Script completo da semana**PRÁTICO · LABORATÓRIO**

Entregue um script .sql com: CREATE TABLE produtos(id, nome, preco DECIMAL(7,2), estoque INT); 5 INSERT; 2 UPDATE com WHERE; 1 DELETE com WHERE; e 3 SELECT (todos os campos; só nome e preco; só os com estoque > 0).

RESULTADO ESPERADO

Script executável sem erros, com WHERE em todas as manipulações e SELECT de conferência antes delas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção automatizada: execute o script de cada dupla em um banco limpo de teste.

EXERCÍCIO 2 — Manipulação pelo VB6**PRÁTICO · LABORATÓRIO**

Formulário com 2 botões: 'Cadastrar' (InputBox nome e nota > InserirAluno) e 'Corrigir nota' (InputBox id e nova nota > SubirNota). Mostre MsgBox de confirmação.

RESULTADO ESPERADO

Inserções e updates refletindo no banco (conferir via cliente MySQL); MsgBox após cada operação.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir no projetor: alterar no VB6 e mostrar a mudança no cliente MySQL ao vivo (efeito cliente/servidor).

EXERCÍCIO 3 — Caça ao erro SQL**ESCRITO**

Cada comando abaixo tem 1 erro. Identifique e corrija: (a) INSERT alunos VALUES ('Ana'); (b) UPDATE alunos SET nota = 9; (c) SELECT nome FROM alunos WHERE nota = '8,5'; (d) DELETE FROM alunos WHERE nome = Ana.

RESULTADO ESPERADO

(a) faltam colunas/valores completos; (b) falta WHERE; (c) Decimal usa ponto 8.5 sem aspas de texto; (d) texto sem aspas simples 'Ana'.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção dialogada; reforce a regra do WHERE e o uso de aspas simples em strings SQL.

TESTE RÁPIDO — MÓDULO 3: SQL: fundamentos e manipulação de dados

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. CREATE TABLE pertence a qual família do SQL?

- (A) DML
- (B) DDL
- (C) DCL
- (D) TCL

RESPOSTA: B

DDL define estrutura; DML manipula dados.

2. PRIMARY KEY AUTO_INCREMENT garante:

- (A) valores duplicados permitidos
- (B) identificador único gerado automaticamente
- (C) ordenação alfabética
- (D) criptografia da coluna

RESPOSTA: B

Chave única, não nula, numerada pelo servidor.

3. UPDATE sem WHERE:

- (A) altera a primeira linha
- (B) altera todas as linhas da tabela
- (C) dá erro de sintaxe
- (D) não faz nada

RESPOSTA: B

Sem filtro, o comando vale para todas as linhas — por isso o SELECT de conferência antes.

4. No VB6, enviamos um comando SQL ao servidor com:

- (A) cn.Execute
- (B) Printer.Print
- (C) Form_Load
- (D) MsgBox

RESPOSTA: A

Connection.Execute; consultas devolvem Recordset.

5. Strings em SQL são escritas com:

- (A) aspas duplas
- (B) aspas simples
- (C) crases
- (D) sem aspas

RESPOSTA: B

'Ana Silva' — aspas simples; aspas duplas são do VB6.

MÓDULO 4 · AULAS 21 A 24 · SEMANA 6

SQL — consultas e filtros

PARTE II — LINGUAGEM SQL

OBJETIVOS DE APRENDIZAGEM

- ✓ Filtrar linhas com WHERE (comparação, BETWEEN, LIKE, IN).
- ✓ Ordenar resultados com ORDER BY (ASC/DESC).
- ✓ Resumir dados com funções de agregação (COUNT, SUM, AVG, MAX, MIN) e GROUP BY.
- ✓ Aplicar consultas em cenários reais de dados (estudo de caso).

AULA 6

Consultas que respondem perguntas

WHERE: a pergunta certa

Filtros essenciais

SQL

```
SELECT nome, nota FROM alunos WHERE nota >= 7;
SELECT nome FROM alunos WHERE curso = 'Informática' AND nota >= 7;
SELECT nome FROM alunos WHERE nome LIKE 'A%'; -- começa com A
SELECT nome FROM alunos WHERE nota BETWEEN 5 AND 7;
SELECT nome FROM alunos WHERE curso IN ('Redes', 'Informática');
```

ORDER BY: a apresentação do resultado

Ordenação

SQL

```
SELECT nome, nota FROM alunos ORDER BY nota DESC; -- maiores primeiro
SELECT nome, nota FROM alunos ORDER BY nome ASC; -- A → Z
```

Agregações: o resumo da ópera

COUNT, SUM, AVG, MAX, MIN + GROUP BY

SQL

```
SELECT COUNT(*) AS total FROM alunos;
SELECT AVG(nota) AS media FROM alunos;
SELECT MAX(nota), MIN(nota) FROM alunos;

SELECT curso, COUNT(*) AS qtd, AVG(nota) AS media
FROM alunos
GROUP BY curso
ORDER BY media DESC;
```

GROUP BY responde “por grupo”

Sem GROUP BY, as agregações resumem a tabela inteira. Com GROUP BY curso, o MySQL separa os grupos e calcula COUNT/AVG **dentro de cada grupo** — é assim que nascem relatórios gerenciais.

Estudo de caso: a loja

Com a tabela vendas(id, produto, qtd, valor, datavenda): quantas vendas por produto? Faturamento total? Melhor mês? Cada pergunta vira uma consulta — e depois vira relatório no módulo 13.

Perguntas de negócio em SQL

SQL

```
SELECT produto, SUM(qtd) AS unidades, SUM(qtd * valor) AS faturamento
FROM vendas
GROUP BY produto
ORDER BY faturamento DESC;
```

EXERCÍCIOS — MÓDULO 4

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Dez perguntas, dez consultas

PRÁTICO · LABORATÓRIO

No banco escola (alunos, produtos, vendas): escreva e execute 10 consultas: 3 com WHERE simples, 2 com LIKE/BETWEEN, 2 com ORDER BY, 3 com agregação (uma delas com GROUP BY). Salve no script consultas_s06.sql.

RESULTADO ESPERADO

Script com 10 SELECT corretos e resultados conferidos no caderno.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção em duplas trocadas: cada dupla executa o script da outra e valida resultados.

EXERCÍCIO 2 — Consultas no VB6

PRÁTICO · LABORATÓRIO

Formulário com ListBox e botão ‘Listar aprovados’: use cn.Execute com SELECT ... WHERE nota >= 7 ORDER BY nome e preencha o ListBox percorrendo o Recordset (Do While Not rs.EOF).

RESULTADO ESPERADO

ListBox populado em ordem alfabética apenas com aprovados; rs.MoveNext e rs.Close presentes.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Este é o embrião do módulo 11: valorize o laço Do While Not rs.EOF como padrão ouro.

EXERCÍCIO 3 — Relatório em papel de uma consulta

ESCRITO

Escolha uma consulta com GROUP BY do exercício 1 e desenhe no caderno a tabela de resultado (colunas e linhas) exatamente como o MySQL devolveria.

RESULTADO ESPERADO

Tabela desenhada com os grupos e valores coerentes com os dados inseridos.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferência cruzada com a saída real no cliente MySQL; discute interpretação de resultado, não só sintaxe.

TESTE RÁPIDO — MÓDULO 4: SQL — consultas e filtros

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Para filtrar nomes que começam com 'A':

- (A) WHERE nome = 'A'
- (B) WHERE nome LIKE 'A%'
- (C) WHERE nome IN 'A'
- (D) WHERE LEFT(nome) = A

RESPOSTA: B

LIKE com curinga % casa padrões de texto.

2. ORDER BY nota DESC exibe:

- (A) das menores para as maiores
- (B) das maiores para as menores
- (C) em ordem de inserção
- (D) aleatoriamente

RESPOSTA: B

DESC = decrescente; ASC (padrão) = crescente.

3. COUNT(*) com GROUP BY curso retorna:

- (A) o total geral de alunos
- (B) a quantidade de alunos por curso
- (C) o número de cursos existentes apenas
- (D) a média por curso

RESPOSTA: B

Agregação calculada dentro de cada grupo definido pelo GROUP BY.

4. BETWEEN 5 AND 7 equivale a:

- (A) nota > 5 AND nota < 7
- (B) nota >= 5 AND nota <= 7
- (C) nota IN (5,7)
- (D) nota = 5 OR nota = 7

RESPOSTA: B

BETWEEN inclui as pontas (fechado nos extremos).

5. No VB6, o resultado de um SELECT chega como:

- (A) MsgBox
- (B) Recordset (ADO)
- (C) arquivo .txt
- (D) Printer

RESPOSTA: B

ADODB.Recordset percorrido com Do While Not rs.EOF.

AVALIAÇÃO À VISTA

AVALIAÇÃO 1 (A1 — 30 pts): trabalho “site da empresa fictícia” (10 pts) + atividades AVA (5 pts) + teste escrito-prático (15 pts). Enunciado completo e rubrica na seção AVALIAÇÕES.

PARTE III

PROGRAMAÇÃO ESTRUTURADA E ESTRUTURAS DE DADOS

AULAS 25 A 40 · SEMANAS 7 A 10

Módulo 5 — Programação estruturada no VB6

Módulo 6 — Vetores e matrizes

Módulo 7 — Registros: agrupando informações relacionadas

MÓDULO 5 · AULAS 25 A 28 · SEMANA 7**Programação estruturada no VB6**

PARTE III — PROGRAMAÇÃO ESTRUTURADA E ESTRUTURAS DE DADOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Aplicar sequência, seleção (If/Select Case) e repetição (For/Do/While) no VB6.
- ✓ Escolher a estrutura de controle adequada a cada problema.
- ✓ Organizar o código com indentação, comentários e nomes claros.
- ✓ Resolver problemas clássicos de lógica com laços e decisões.

AULA 7**Seleção e repetição: o cânone estruturado****Seleção: If completo e Select Case**

If / ElseIf / Else — situação do aluno

VB

```
Dim nota As Single
nota = Val(InputBox("Digite a nota:"))

If nota >= 7 Then
Print nota; "- APROVADO"
ElseIf nota >= 5 Then
Print nota; "- RECUPERAÇÃO"
Else
Print nota; "- REPROVADO"
End If
```

Select Case — menu de opções

VB

```
Select Case opcao
Case 1
Call CadastrarAluno
Case 2
Call ListarAlunos
Case 3
Call GerarRelatorio
Case Else
MsgBox "Opção inválida", vbExclamation
End Select
```

Repetição: For, Do While e Do Until

Os três laços, lado a lado

VB

```
' FOR: quando se sabe quantas voltas
For i = 1 To 10
Print i;
Next i

' DO WHILE: testa antes, repete enquanto verdadeiro
Do While nota <> -1
soma = soma + nota
nota = Val(InputBox("Nota (-1 termina):"))
Loop

' DO UNTIL: repete até a condição virar verdadeira
Do Until senha = "1234"
senha = InputBox("Senha:")
Loop
```

Qual laço escolher?

For: número de voltas conhecido (contar, percorrer vetor). **Do While:** repetir enquanto uma condição valer (ler até sentinela). **Do Until:** repetir até alcançar um estado (validar entrada). **While...Wend:** existe no VB6, mas evitamos — o Do é mais claro e flexível.

Organização lógica do código

- Indentação de 4 espaços dentro de cada bloco (If/For/Select);
- Comentários ' explicando o PORQUÊ, não o óbvio;
- Uma responsabilidade por Sub (nome em verbo: CalcularMedia, ListarAprovados);
- Variáveis declaradas no topo com Dim e tipo explícito;
- Sentinela documentada: -1 termina a leitura (combinado com o usuário).

ATENÇÃO

Laço infinito no VB6 trava a IDE: se acontecer, Ctrl+Break (ou o botão End) e procure a variável que não avança. Todo Do/While precisa de um caminho que torne a condição falsa.

EXERCÍCIOS — MÓDULO 5

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Calculadora estruturada

PRÁTICO · LABORATÓRIO

Formulário com dois Text (n1, n2) e um Combo de operação (+, -, *, /): ao clicar em =, use Select Case para calcular e Label para o resultado; trate divisão por zero com MsgBox (If).

RESULTADO ESPERADO

Calculadora funcional com Select Case e tratamento de divisão por zero sem travar.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir uso de Val() nos Text e Case Else para operação inválida.

EXERCÍCIO 2 — Leitura até sentinela**PRÁTICO · LABORATÓRIO**

Com Do While, leia notas até o usuário digitar -1; ao final mostre quantidade, soma, média, maior e menor nota (variáveis de acompanhamento).

RESULTADO ESPERADO

Laço com sentinela correta e cinco estatísticas certas (cuidado: -1 não entra na conta).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Erro clássico: contar o -1. Peça teste com 0 notas digitadas (média não pode dar erro de divisão).

EXERCÍCIO 3 — Refatoração dirigida**ESCRITO**

O código abaixo funciona, mas está desorganizado. Reescreva-o com indentação, comentários de porquê, nomes claros e um Select Case no lugar dos Ifs encadeados de menu: (código com 6 Ifs aninhados sem indentação fornecido no quadro/apostila).

RESULTADO ESPERADO

Versão legível: Select Case para o menu, Subs nomeados por responsabilidade, indentação consistente.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção coletiva no projetor com votação: qual versão é mais fácil de manter? Introduz o valor da modularização (próxima semana 12).

TESTE RÁPIDO — MÓDULO 5: Programação estruturada no VB6

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Para um menu com opções fixas 1, 2 e 3, a estrutura mais legível é:

- (A) If aninhados
- (B) Select Case
- (C) Do Until
- (D) For

RESPOSTA: B

Select Case compara um valor com casos fixos — menus ficam limpos.

2. Do While repete:

- (A) sempre ao menos uma vez
- (B) enquanto a condição for verdadeira (teste antes)
- (C) até a condição virar verdadeira
- (D) um número fixo de voltas

RESPOSTA: B

Testa antes de cada volta; pode executar zero vezes.

3. Um laço infinito geralmente acontece por:

- (A) excesso de comentários
- (B) variável de controle que nunca torna a condição falsa
- (C) usar Print dentro do laço
- (D) declarar Dim dentro do laço

RESPOSTA: B

Sem avanço/saída, a condição nunca muda — Ctrl+Break e corrija.

4. Sentinela é:

- (A) um erro de execução
- (B) um valor combinado que encerra a leitura (ex.: -1)
- (C) uma variável global
- (D) um tipo de laço

RESPOSTA: B

Padrão de leitura até sentinela com Do While.

5. Val(InputBox(...)) serve para:

- (A) exibir texto
- (B) converter o texto digitado em número
- (C) validar senha
- (D) fechar a janela

RESPOSTA: B

InputBox retorna String; Val converte para número.

MÓDULO 6 · AULAS 29 A 36 · SEMANAS 8 A 9

Vetores e matrizes

PARTE III — PROGRAMAÇÃO ESTRUTURADA E ESTRUTURAS DE DADOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Declarar, preencher, ler e processar vetores (arrays unidimensionais).
- ✓ Trabalhar com matrizes (duas dimensões) e laços aninhados.
- ✓ Resolver problemas clássicos: média, maior/menor, busca, somas por linha/coluna.
- ✓ Combinar vetores/matrizes com laços e decisões de forma organizada.

AULA 8**Vetores: muitas caixinhas, um nome só****Declaração e índices**

Vetor de 10 notas

VB

```
Dim notas(1 To 10) As Single
Dim i As Integer

' preenchimento
For i = 1 To 10
    notas(i) = Val(InputBox("Nota " & i & ":"))
Next i

' leitura/processamento
Dim soma As Single
For i = 1 To 10
    soma = soma + notas(i)
Next i
Print "Média: "; soma / 10
```

Índice é o endereço da caixinha

notas(1) é a PRIMEIRA posição quando declaramos (1 To 10). O índice pode vir de uma variável ou expressão: notas(i), notas(posicao) — é isso que permite percorrer tudo com um laço.

Processamentos clássicos

Maior, menor e busca

VB

```
Dim maior As Single, menor As Single, pos As Integer
maior = notas(1): menor = notas(1)
For i = 2 To 10
    If notas(i) > maior Then maior = notas(i)
    If notas(i) < menor Then menor = notas(i)
Next i

' busca: em que posição está a nota 10?
pos = 0
For i = 1 To 10
    If notas(i) = 10 Then pos = i
Next i
If pos > 0 Then Print "Acho na posição "; pos
```

DICA

Padrão de ouro: inicialize maior/menor com o PRIMEIRO elemento (não com 0!) e comece o laço de comparação no segundo. Funciona até com notas negativas.

AULA 9**Matrizes: linhas e colunas****Declaração e acesso**

Matriz 3×4 e laços aninhados

VB

```
Dim m(1 To 3, 1 To 4) As Integer
Dim i As Integer, j As Integer

' preenchimento
For i = 1 To 3
    For j = 1 To 4
        m(i, j) = i * j
    Next j
Next i

' soma por linha
For i = 1 To 3
    Dim sl As Integer: sl = 0
    For j = 1 To 4
        sl = sl + m(i, j)
    Next j
    Print "Linha "; i; " soma "; sl
Next i
```

Laço de fora = linha; laço de dentro = coluna

Para varrer uma matriz, o laço externo percorre as **linhas** (i) e o interno, as **colunas** (j). Somar por linha: zere o acumulador DENTRO do laço de linhas, antes do laço de colunas.

Aplicações

- Tabela de notas por aluno × bimestre;
- Tabuada 1–9 em grade;
- Mapa de estoque por loja × produto;
- Transposição e comparação de matrizes (desafio).

ATENÇÃO

Índice fora da faixa (m(4,1) em matriz 3×4) gera erro de execução "subscript out of range". Confira os limites do For com a declaração.

EXERCÍCIOS — MÓDULO 6

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Vetor completo

PRÁTICO · LABORATÓRIO

Leia 8 temperaturas em um vetor; mostre: média, maior, menor, quantas acima da média e a posição da maior.

RESULTADO ESPERADO

Cinco saídas corretas com um único vetor e laços separados por responsabilidade.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir inicialização maior/menor e a contagem acima da média (laço extra).

EXERCÍCIO 2 — Matriz de notas

PRÁTICO · LABORATÓRIO

Matriz 4 alunos × 3 bimestres: preencha por InputBox; mostre a média de cada aluno (por linha), a média de cada bimestre (por coluna) e a maior nota com sua posição.

RESULTADO ESPERADO

Médias por linha e coluna corretas + maior nota com (i, j).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Ponto de atenção: dois acumuladores diferentes (linha × coluna) e zerar no lugar certo. Teste cruzado entre duplas com gabarito manual.

EXERCÍCIO 3 — Busca no vetor

ESCRITO

Escreva de memória o padrão de busca: ler um valor e informar a posição dele no vetor de 10 notas (ou 'não encontrado'). Depois explique por que `pos = 0` antes do laço é essencial.

RESULTADO ESPERADO

Laço com `if` comparando e guardando `pos`; explicação: 0 funciona como flag de "não achou", pois posições válidas começam em 1.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Reforce o conceito de flag/sentinela de busca — reaparece em arquivos e Recordset.

TESTE RÁPIDO — MÓDULO 6: Vetores e matrizes

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Dim `v(1 To 5)` As Integer cria:

- (A) 5 variáveis chamadas `v1..v5`
- (B) um vetor com posições 1 a 5
- (C) uma matriz 5×5
- (D) um vetor de textos

RESPOSTA: B

Array unidimensional com índices explícitos de 1 a 5.

2. Para percorrer todo o vetor, o índice usado no acesso deve ser:

- (A) fixo (`v(1)`)
- (B) a variável do laço (`v(i)`)
- (C) sempre 0
- (D) um `InputBox`

RESPOSTA: B

Índice variável é o que permite o percurso com `For`.

3. Na varredura de matriz, o laço EXTERNO normalmente percorre:

- (A) as colunas
- (B) as linhas
- (C) as diagonais
- (D) os índices aleatórios

RESPOSTA: B

Externo = linhas (`i`); interno = colunas (`j`).

4. 'Subscript out of range' significa:

- (A) variável não declarada
- (B) índice fora dos limites declarados
- (C) divisão por zero
- (D) arquivo não encontrado

RESPOSTA: B

Acessar $m(4,1)$ em matriz 3×4 , por exemplo.

5. Para achar o maior valor de um vetor de notas (que podem ser negativas), inicialize maior com:

- (A) 0
- (B) 10
- (C) o primeiro elemento do vetor
- (D) um número aleatório

RESPOSTA: C

Iniciar no 1º elemento garante corretude para qualquer faixa.

AVALIAÇÃO À VISTA

Semana 8: TESTE ESCRITO-PRÁTICO da A1 (15 pts) sobre Web e HTML. Modelo completo na seção AVALIAÇÕES.

MÓDULO 7 · AULAS 37 A 40 · SEMANA 10**Registros: agrupando informações relacionadas**

PARTE III — PROGRAMAÇÃO ESTRUTURADA E ESTRUTURAS DE DADOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Definir tipos estruturados com Type/End Type.
- ✓ Declarar variáveis e vetores de registros.
- ✓ Acessar campos com o operador ponto e processar coleções de registros.
- ✓ Modelar entidades reais (aluno, produto) com registros.

AULA 10**Registros (Type): a ficha dentro do programa****Definindo um tipo**

Module Tipos.bas

VB

```
Public Type Aluno
    nome As String * 40
    matricula As Integer
    nota As Single
End Type

Public Type Produto
    codigo As Integer
    descricao As String * 30
    preco As Currency
    estoque As Integer
End Type
```

Usando registros e vetores de registros

Ficha e coleção

VB

```
Dim a As Aluno
a.nome = "Ana Silva"
a.matricula = 101
a.nota = 8.5
Print a.nome; " - "; a.nota

' coleção: vetor de fichas
Dim turma(1 To 30) As Aluno
Dim i As Integer
For i = 1 To 30
    turma(i).nome = InputBox("Nome do aluno " & i)
    turma(i).nota = Val(InputBox("Nota:"))
Next i
```

Registro = linha de tabela dentro do programa

Um registro agrupa campos de tipos diferentes sob um nome só — exatamente como uma LINHA de tabela no banco. Vetor de registros = tabela em memória. É a ponte mental entre as estruturas do VB6 e as tabelas do MySQL.

Processamentos típicos

Aprovados da turma (vetor de registros)

VB

```
For i = 1 To 30
    If turma(i).nota >= 7 Then
        Print turma(i).nome; " aprovado com "; turma(i).nota
    End If
Next i
```

DICA

String * 40 fixa o tamanho do campo (herança dos arquivos de registro fixo). Para textos de tamanho livre em memória, use String simples — discutimos o trade-off na semana de arquivos.

EXERCÍCIOS — MÓDULO 7

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Ficha do produto

PRÁTICO · LABORATÓRIO

Defina o Type Produto (codigo, descricao, preco, estoque); cadastre 5 produtos em um vetor; liste os produtos com estoque < 5 (reposição) e o valor total do estoque (preco × estoque somado).

RESULTADO ESPERADO

Type correto, cadastro em vetor e duas saídas (reposição + valor total) conferindo.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir uso de Currency para preço e o acumulador do valor total.

EXERCÍCIO 2 — Turma completa**PRÁTICO · LABORATÓRIO**

Vetor de 10 registros Aluno: leia nome/matricula/nota; mostre aprovados, reprovados, média da turma e o nome do aluno de maior nota.

RESULTADO ESPERADO

Quatro saídas corretas percorrendo o vetor de registros uma ou mais vezes.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Incentive Subs separados (ListarAprovados, MediaTurma...) — prepara a semana 12.

EXERCÍCIO 3 — Modelagem no papel**ESCRITO**

Para um controle de biblioteca, defina no caderno os Types Livro e Emprestimo (campos e tipos) e explique como um vetor de cada um representaria as 'tabelas' do sistema.

RESULTADO ESPERADO

Types coerentes (ex.: Livro: codigo, titulo, autor, disponível; Emprestimo: codigoLivro, matriculaAluno, data) e explicação da relação entre vetores.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Essa modelagem vira o banco MySQL do projeto integrador — guarde os melhores modelos.

TESTE RÁPIDO — MÓDULO 7: Registros: agrupando informações relacionadas

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Public Type Aluno ... End Type define:

- (A) um formulário
- (B) um tipo estruturado (registro)
- (C) um banco de dados
- (D) uma Function

RESPOSTA: B

Registro: campos de tipos diferentes sob um nome.

2. O acesso ao campo nota do i-ésimo aluno de um vetor de registros é:

- (A) turma.nota(i)
- (B) turma(i).nota
- (C) nota.turma(i)
- (D) turma[i,nota]

RESPOSTA: B

Primeiro o índice do vetor, depois o ponto e o campo.

3. Um vetor de registros equivale, no mundo do banco, a:

- (A) uma coluna
- (B) uma tabela em memória (conjunto de linhas)
- (C) um índice
- (D) uma view

RESPOSTA: B

Cada registro = linha; o vetor = conjunto de linhas.

4. String * 40 em um Type significa:

- (A) texto de até 40 caracteres com tamanho fixo
- (B) 40 strings diferentes
- (C) texto ilimitado
- (D) número de 40 dígitos

RESPOSTA: A

Campo de tamanho fixo — útil em arquivos de registro fixo.

5. Para somar o valor total do estoque (preço × estoque) percorrendo um vetor de Produtos, usamos:

- (A) um laço com acumulador: total = total + p(i).preço * p(i).estoque
- (B) SUM(preço) como no SQL
- (C) Print p.preço * p.estoque sem laço
- (D) Count(p)

RESPOSTA: A

Em memória, agregações são laços com acumulador — no banco, SQL.

PARTE IV

STRINGS, MODULARIZAÇÃO E ARQUIVOS

AULAS 41 A 56 • SEMANAS 11 A 14

Módulo 8 — Variáveis do tipo String

Módulo 9 — Modularização: Sub, Function e Modules

Módulo 10 — Arquivos e textos como transferência de dados

MÓDULO 8 · AULAS 41 A 44 · SEMANA 11

Variáveis do tipo String

PARTE IV — STRINGS, MODULARIZAÇÃO E ARQUIVOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Armazenar, ler e exibir textos com variáveis String.
- ✓ Aplicar as funções de manipulação: Len, Mid\$, Left\$, Right\$, UCase\$, LCase\$, Trim\$, InStr, Replace.
- ✓ Validar e normalizar entradas de texto (caixa, espaços, padrões).
- ✓ Combinar Strings com números (Val, Str, Format) em saídas legíveis.

AULA 11**String: o texto como dado****O essencial das Strings**

Funções de manipulação

VB

```
Dim s As String
s = " Linguagem de Programação II "

Print Len(s) ' 33 (conta espaços)
Print Trim$(s) ' sem espaços nas pontas
Print UCase$(Trim$(s)) ' CAIXA ALTA
Print Left$(s, 5) ' " Lin"
Print Right$(s, 2) ' " "... cuidado com espaços!
Print Mid$(s, 3, 9) ' "Linguagem"
Print InStr(s, "Program") ' posição onde começa (16)
Print Replace$(s, "II", "III")
```

Posições começam em 1

No VB6, Mid\$/Left\$/Right\$ contam a partir de **1** (não de 0). InStr retorna 0 quando não encontra — use como flag: If InStr(...) > 0 Then.

Validação e normalização de entrada

Padronizar nome digitado

VB

```
Dim nome As String
nome = Trim$(InputBox("Nome completo:"))

If Len(nome) < 5 Then
    MsgBox "Nome muito curto!"
ElseIf InStr(nome, " ") = 0 Then
    MsgBox "Digite nome e sobrenome!"
Else
    nome = UCase$(nome) ' padroniza caixa
    Print nome
End If
```

Strings x números em saídas legíveis

Format e conversões

VB

```
Dim media As Single, txt As String
media = 7.25
txt = "Média: " & Format(media, "0.00") ' Média: 7,25
Print txt
Print "Hoje: " & Format(Date, "dd/mm/yyyy")
Print Str$(42) & " | " & Val("42 anos") ' 42 | 42
```

DICA

Montar mensagens com & e Format é o que separa saída de iniciante ("7,250000E+00") de saída profissional ("7,25"). Relatórios (semana 19) dependem disso.

EXERCÍCIOS — MÓDULO 8

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Caixa de ferramentas de String

PRÁTICO · LABORATÓRIO

Leia uma frase e mostre: comprimento sem espaços nas pontas; frase em caixa alta e baixa; primeira e última palavra; posição da palavra 'VB'; frase sem a palavra 'não' (Replace).

RESULTADO ESPERADO

Seis saídas corretas usando as funções da aula (última palavra via InStrRev ou Right complito aceito).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Discuta soluções diferentes para a última palavra — vale qualquer abordagem correta.

EXERCÍCIO 2 — Validador de matrículas

PRÁTICO · LABORATÓRIO

Matrícula padrão: 2 letras + 4 números (ex.: AB1234). Leia uma string e valide formato e comprimento com Len/Mid\$/IsNumeric; mostre OK ou o motivo da recusa.

RESULTADO ESPERADO

Validação por posição: Mid\$(s,1,2) letras (Not IsNumeric) e Mid\$(s,3,4) numérica; mensagens específicas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Excelente para fixar Mid\$ por posição; teste com casos inválidos variados.

EXERCÍCIO 3 — Recibo formatado**PRÁTICO · LABORATÓRIO**

Monte um 'recibo' em um Label multilinha: nome em caixa alta, data dd/mm/yyyy, valor Format(v, "R\$ ###0.00") e linha separadora String(40, "-").

RESULTADO ESPERADO

Recibo legível com todos os Format corretos e separador.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Ponte direta para relatórios (semana 19): elogie os recibos mais bem alinhados.

TESTE RÁPIDO — MÓDULO 8: Variáveis do tipo String

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Mid\$("LINGUAGEM", 3, 4) retorna:

- (A) "LING"
- (B) "NGUA"
- (C) "GUAG"
- (D) "AGEM"

RESPOSTA: B

Começa na posição 3 e pega 4 caracteres: N G U A.

2. InStr retorna 0 quando:

- (A) encontra no início
- (B) não encontra o trecho
- (C) o texto está vazio de espaços
- (D) o texto é numérico

RESPOSTA: B

0 = não achou; >0 = posição inicial do trecho.

3. Para remover espaços das pontas de um texto digitado:

- (A) Trim\$
- (B) Mid\$
- (C) Left\$
- (D) Replace\$ com " "

RESPOSTA: A

Trim\$ corta espaços à esquerda e à direita.

4. No VB6, as posições de String contam a partir de:

- (A) 0
- (B) 1
- (C) -1
- (D) depende do Option Base

RESPOSTA: B

Funções de texto iniciam em 1.

5. Format(7.25, "0.00") produz:

- (A) 7,25 como texto formatado
- (B) 725
- (C) 7.250000
- (D) erro

RESPOSTA: A

Saída legível com duas casas — padrão de relatórios.

MÓDULO 9 · AULAS 45 A 48 · SEMANA 12

Modularização: Sub, Function e Modules

PARTE IV — STRINGS, MODULARIZAÇÃO E ARQUIVOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Dividir programas em Sub e Function com responsabilidades únicas.
- ✓ Usar parâmetros (ByVal/ByRef) e retorno de valores.
- ✓ Organizar código em Modules (.bas) com escopos Public/Private.
- ✓ Refatorar código monolítico em módulos reutilizáveis.

AULA 12

Modularização: dividir para conquistar

Sub × Function (revisitando com profundidade)

Modulo Calculos.bas

VB

```
Public Function Media(n1 As Single, n2 As Single) As Single
Media = (n1 + n2) / 2 ' retorno = nome da Function
End Function

Public Function Situacao(m As Single) As String
If m >= 7 Then
Situacao = "APROVADO"
ElseIf m >= 5 Then
Situacao = "RECUPERAÇÃO"
Else
Situacao = "REPROVADO"
End If
End Function

Public Sub MostrarBoletim(n1 As Single, n2 As Single)
Dim m As Single
m = Media(n1, n2)
MsgBox "Média " & Format(m, "0.0") & " - " & Situacao(m)
End Sub
```

Parâmetros: ByVal e ByRef

Efeito colateral controlado

VB

```
Public Sub Dobrar(ByVal x As Integer, ByRef y As Integer)
    x = x * 2 ' mexe só na CÓPIA local
    y = y * 2 ' mexe na variável ORIGINAL do chamador
End Sub

Dim a As Integer, b As Integer
a = 5: b = 5
Dobrar a, b
Print a; b ' 5 10
```

ByVal protege; ByRef compartilha

ByVal: o parâmetro recebe uma cópia — o original não muda (padrão seguro). **ByRef:** o parâmetro é a própria variável do chamador — use só quando a Sub PRECISA devolver algo por parâmetro. No VB6, parâmetros são ByRef por padrão: escreva ByVal explicitamente por segurança.

Escopo e organização em Modules

- **Public** em Module (.bas): visível no projeto inteiro (conexão, cálculos, SQL);
- **Private:** só dentro do módulo/formulário (detalhes internos);
- Variável declarada dentro de Sub = **local**; no topo do Module com Public = **global** (use com parcimônia!);
- Nomes em verbo + objeto: CalcularMedia, ListarAprovados, GravarArquivo;
- Um módulo por assunto: Conexao.bas, Calculos.bas, Arquivos.bas, Relatorios.bas.

Refatoração: do monólito aos módulos

Pegue o exercício 'monolítico' da semana 7 (menu com tudo dentro de um clique) e divida: um Module de regras (Functions puras), um Module de saída (Subs de Print/MsgBox) e o formulário só orquestrando chamadas. Mesma funcionalidade, manutenção dez vezes mais fácil.

EXERCÍCIOS — MÓDULO 9

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Biblioteca de funções**PRÁTICO · LABORATÓRIO**

Crie Calculos.bas com: Media(n1,n2), MaiorDe3(a,b,c), EhPar(n) e PorExtensoDezena(n) (10..19). Teste todas em um formulário com botões.

RESULTADO ESPERADO

Quatro Functions puras (sem MsgBox dentro) retornando corretamente; formulário só exibe resultados.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Regra de ouro: Function não conversa com usuário (sem InputBox/MsgBox) — quem chama orquestra.

EXERCÍCIO 2 — ByVal × ByRef na prática**PRÁTICO · LABORATÓRIO**

Implemente Sub Trocar(ByRef a, ByRef b) que inverte dois valores e Sub TrocarCopia(ByVal a, ByVal b) que não muda nada fora; demonstre a diferença com Print.

RESULTADO ESPERADO

Demonstração correta: Trocar altera as variáveis do chamador; TrocarCopia não.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Pergunta de fixação: por que declaramos ByVal por padrão no curso?

EXERCÍCIO 3 — Refatoração do monólito**PRÁTICO · LABORATÓRIO**

Refatore o menu monolítico da semana 7 em: Regras.bas (Functions), Saida.bas (Subs) e formulário orquestrador com Select Case chamando as Subs.

RESULTADO ESPERADO

Três camadas claras; nenhuma regra de negócio dentro do formulário.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Checklist: nenhuma Function com MsgBox? nenhum Print em Regras.bas? nomes em verbo?

TESTE RÁPIDO — MÓDULO 9: Modularização: Sub, Function e Modules

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. O valor de retorno de uma Function no VB6 é atribuído:

- (A) à palavra Return
- (B) ao próprio nome da Function
- (C) a uma variável global obrigatória
- (D) ao parâmetro ByRef sempre

RESPOSTA: B

Media = (n1+n2)/2 — o nome da Function recebe o retorno.

2. ByVal significa que o parâmetro:

- (A) é a própria variável do chamador
- (B) recebe uma cópia (original protegido)
- (C) deve ser numérico
- (D) é opcional

RESPOSTA: B

Cópia local; por segurança, declare ByVal explicitamente.

3. Modules .bas com rotinas Public servem para:

- (A) guardar telas
- (B) compartilhar regras entre formulários do projeto
- (C) substituir o banco de dados
- (D) compilar mais rápido

RESPOSTA: B

Camada de regras reutilizável em todo o projeto.

4. Uma Function 'pura', no padrão do curso:

- (A) usa MsgBox para pedir dados
- (B) não conversa com o usuário: só recebe e retorna
- (C) imprime com Print
- (D) grava arquivo

RESPOSTA: B

Entrada por parâmetros, saída por retorno; I/O fica com quem chama.

5. Variável declarada dentro de uma Sub é:

- (A) global
- (B) local à execução da Sub
- (C) pública do módulo
- (D) estática permanente

RESPOSTA: B

Escopo local: nasce e morre a cada chamada.

MÓDULO 10 · AULAS 49 A 56 · SEMANAS 13 A 14

Arquivos e textos como transferência de dados

PARTE IV — STRINGS, MODULARIZAÇÃO E ARQUIVOS

OBJETIVOS DE APRENDIZAGEM

- ✓ Gravar e ler arquivos texto com Open/Close, Print #, Input # e Line Input #.
- ✓ Usar modos For Output, For Input e For Append corretamente.
- ✓ Detectar fim de arquivo com EOF e numerar arquivos com FreeFile.
- ✓ Integrar arquivos aos dados do programa: importação/exportação em fluxo controlado.

AULA 13

Arquivos texto: gravar e ler

O ciclo de vida de um arquivo

Gravar (Output) e acrescentar (Append)

VB

```

Dim n As Integer
n = FreeFile ' número de arquivo livre
Open "c:\lp2\dados\alunos.txt" For Output As #n
Print #n, "Ana Silva;101;8.5" ' linha com campos separados
Print #n, "Beto Souza;102;6.0"
Close #n

Open "c:\lp2\dados\alunos.txt" For Append As #n
Print #n, "Carla Dias;103;7.5" ' acrescenta sem apagar
Close #n

```

Ler até o fim (EOF)

VB

```

Dim n As Integer, linha As String
n = FreeFile
Open "c:\lp2\dados\alunos.txt" For Input As #n
Do While Not EOF(n)
Line Input #n, linha
List1.AddItem linha
Loop
Close #n

```

Modos de abertura

Modo	Efeito	Quando usar
For Output	cria/substitui o arquivo vazio	gerar um arquivo novo
For Append	abre e posiciona no final	acrescentar sem perder
For Input	somente leitura	importar/consultar

EOF e FreeFile

EOF(n) = True quando a leitura chegou ao fim (fim do arquivo). **FreeFile** devolve um número de arquivo livre — evita conflito quando vários arquivos ficam abertos ao mesmo tempo. Par obrigatório: Open ... Close.

ATENÇÃO

Esquecer o **Close** = arquivo travado e dados perdidos no pior momento. Esquecer que **Output apaga tudo** = tragédia de laboratório: para acrescentar, use Append.

AULA 14**Arquivos + dados: importação/exportação controlada****Do arquivo para o vetor de registros**

Importar alunos.txt para a turma em memória

VB

```
Public Sub ImportarTurma(caminho As String)
    Dim n As Integer, linha As String
    Dim partes() As String
    qt = 0
    n = FreeFile
    Open caminho For Input As #n
    Do While Not EOF(n)
        Line Input #n, linha
        partes = Split(linha, ";") ' nome;matricula;nota
        qt = qt + 1
        turma(qt).nome = partes(0)
        turma(qt).matricula = Val(partes(1))
        turma(qt).nota = Val(partes(2))
    Loop
    Close #n
End Sub
```

Exportar (gravar) a turma de volta

VB

```
Public Sub ExportarTurma(caminho As String)
    Dim n As Integer, i As Integer
    n = FreeFile
    Open caminho For Output As #n
    For i = 1 To qt
        Print #n, turma(i).nome & ";" & turma(i).matricula & _
            "," & Format(turma(i).nota, "0.0")
    Next i
    Close #n
End Sub
```

Fluxo controlado de importação/exportação

1. Validar existência do arquivo antes de abrir (Dir(caminho) <> "");

2. Abrir em Input e ler linha a linha com Split no separador combinado (,);
3. Validar cada linha (qt de partes, tipos com Val/IsNumeric) antes de popular o vetor;
4. Linhas inválidas: contar e reportar ao final (não travar nelas);
5. Exportar com Format para manter padronização de números/datas.

Arquivo = fila de caixas etiquetadas

Cada linha é uma caixa; o separador ; é a etiqueta interna que diz o que há dentro. Importar é abrir caixas uma a uma e conferir o conteúdo; exportar é montar caixas padronizadas e fechar o caminhão (Close).

EXERCÍCIOS — MÓDULO 10

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Diário de classe em arquivo

PRÁTICO · LABORATÓRIO

Grave com Append as chamadas diárias (data;matricula;presença) em diario.txt; crie botão que liste todo o arquivo em um ListBox usando EOF.

RESULTADO ESPERADO

Arquivo crescendo a cada clique (Append) e listagem completa correta.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir se usaram Append (Output apagaria o histórico) e Close em ambos.

EXERCÍCIO 2 — Importação com validação

PRÁTICO · LABORATÓRIO

Importe alunos.txt (nome;matricula;nota) para o vetor turma com validação por linha (3 partes, nota numérica); ao final mostre: importadas, recusadas e média dos importados.

RESULTADO ESPERADO

Relatório de importação correto mesmo com linhas corrompidas no arquivo de teste.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Forneça o arquivo com armadilhas; valorize o relatório de recusadas com motivo.

EXERCÍCIO 3 — Export > Import round-trip

ESCRITO

Explique por que exportar com Format(nota, "0.0") e importar com Val mantém o round-trip íntegro; o que aconteceria sem o Format em notas como 8.25?

RESULTADO ESPERADO

Explicação: Format padroniza casas e separador; sem ele, saídas como 8,250000 poluiriam o arquivo; Val leria só o '8' antes da vírgula em alguns cenários.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Discussão curta que ancora a importância de padronização de dados em trânsito.

TESTE RÁPIDO — MÓDULO 10: Arquivos e textos como transferência de dados

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Para **ACRESCENTAR** linhas sem apagar o conteúdo existente, abre-se com:

- (A) For Output
- (B) For Input
- (C) For Append
- (D) For Random

RESPOSTA: C

Append posiciona no fim; Output recria vazio.

2. EOF(n) retorna True quando:

- (A) o arquivo foi aberto
- (B) a leitura chegou ao fim do arquivo
- (C) houve erro de disco
- (D) o arquivo está vazio de linhas válidas

RESPOSTA: B

Fim de arquivo — condição de parada do Do While.

3. FreeFile serve para:

- (A) liberar memória
- (B) obter um número de arquivo livre
- (C) apagar o arquivo
- (D) fechar todos os arquivos

RESPOSTA: B

Evita conflito de #n com outros arquivos abertos.

4. Split(linha, ";") devolve:

- (A) um texto único
- (B) um vetor com os campos separados por ;
- (C) o primeiro campo apenas
- (D) um registro Type

RESPOSTA: B

Array de partes: partes(0), partes(1)...

5. Esquecer o Close após gravação pode causar:

- (A) nada, o VB6 fecha sozinho sempre
- (B) perda de dados e arquivo travado
- (C) conversão errada de tipos
- (D) erro de sintaxe

RESPOSTA: B

Buffer não descarregado/arquivo bloqueado — feche sempre.

PARTE V

DADOS CLIENTE/SERVIDOR E ADMINISTRAÇÃO DO BANCO

AULAS 57 A 72 · SEMANAS 15 A 18

Módulo 11 — Manipulação de dados Cliente/Servidor (ADO)

Módulo 12 — Backup e recuperação de banco de dados

MÓDULO 11 · AULAS 57 A 64 · SEMANAS 15 A 16

Manipulação de dados Cliente/Servidor (ADO)

PARTE V — DADOS CLIENTE/SERVIDOR E ADMINISTRAÇÃO DO BANCO

OBJETIVOS DE APRENDIZAGEM

- ✓ Explicar o fluxo Cliente/Servidor: aplicação > SQL > SGBD > resultados.
- ✓ Percorrer resultados com Recordset (EOF, MoveNext, campos por nome).
- ✓ Executar CRUD completo (CREATE/INSERT/SELECT/UPDATE/DELETE) pelo VB6.
- ✓ Validar entradas e tratar problemas de conexão/SQL em tempo de execução.

AULA 15

Cliente/Servidor: o fluxo completo com ADO

O fluxo que rege tudo

A aplicação VB6 (cliente) monta o SQL, envia pela conexão; o MySQL (servidor) executa, e devolve: ou um **Recordset** (consulta) ou um **número de linhas afetadas** (manipulação). Entender esse vai-e-vem é entender sistemas reais.

Recordset: percorrendo o resultado

VB

```
Dim rs As ADODB.Recordset
Set rs = cn.Execute("SELECT nome, nota FROM alunos " & _
"WHERE nota >= 7 ORDER BY nome")

Do While Not rs.EOF
List1.AddItem rs!nome & " - " & rs!nota
rs.MoveNext
Loop
rs.Close
Set rs = Nothing
```

O trio sagrado do Recordset

rs.EOF: chegou ao fim? · **rs.MoveNext**: avance uma linha · **rs!campo**: leia o valor da coluna. Sem MoveNext, laço infinito; sem Close, recurso preso no servidor.

CRUD completo pelo programa

Módulo CrudAlunos.bas

VB

```
Public Sub Inserir(nome As String, nota As Single)
cn.Execute "INSERT INTO alunos (nome, nota) VALUES ('" & _
nome & "', " & Format(nota, "0.0") & ")"
End Sub

Public Sub Atualizar(id As Integer, nota As Single)
cn.Execute "UPDATE alunos SET nota = " & Format(nota, "0.0") & _
" WHERE id = " & id
End Sub

Public Sub Excluir(id As Integer)
cn.Execute "DELETE FROM alunos WHERE id = " & id
End Sub

Public Function Existe(id As Integer) As Boolean
Dim rs As ADODB.Recordset
Set rs = cn.Execute("SELECT id FROM alunos WHERE id = " & id)
Existe = Not rs.EOF
rs.Close
End Function
```

Validação e tratamento de problemas

- Validar ANTES de mandar SQL: Len/Trim/Val/IsNumeric no cliente;
- Aspas simples em textos: escapar ' duplicando (") para não quebrar o comando;
- On Error GoTo em toda operação de banco; Err.Number 2000+ = erro de banco/conexão;
- Confirmar exclusões com MsgBox vbYesNo antes do DELETE;
- Mostrar ao usuário mensagem amigável + detalhe técnico no log (Debug.Print).

ATENÇÃO

Injeção de SQL: se o texto digitado entra direto no comando, um usuário mal-intencionado 'fecha as aspas' e injeta comandos. Por isso validamos, escapamos e, em sistemas reais, usamos parâmetros preparados. Consciência desde já.

AULA 16

Cliente/Servidor — prática e validação em grupo

Oficina de CRUD com tela

1. Formulário frmAlunos: Text nome, Text nota, botões Incluir/Alterar/Excluir/Listar;
2. Listar preenche ListBox via Recordset (nome + nota + id entre colchetes);
3. Incluir valida (nome >= 5 caracteres, nota 0–10) antes do INSERT;
4. Alterar/Excluir pedem o id e confirmam com MsgBox vbYesNo;
5. Toda operação termina atualizando a ListBox (refresh).

Testes de robustez (atividade em grupo)

Plano de testes da oficina

Teste	Entrada	Esperado
Nome vazio	(vazio)	MsgBox de validação, nada vai ao banco
Nota fora de faixa	nota = 15	MsgBox, nada vai ao banco
Aspas no nome	O'Connor	INSERT funciona (aspas escapadas)
Excluir inexistente	id = 999	Aviso 'não encontrado' (Existe=False)
Serviço parado	MySQL off	MsgBox amiga de falha de conexão

DICA

Teste em dupla trocada: uma dupla ataca o programa da outra com o plano de testes. Quem acha bug, assina o relatório — caçar bug também é conteúdo.

EXERCÍCIOS — MÓDULO 11

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — CRUD de produtos**PRÁTICO · LABORATÓRIO**

Replique a oficina para a tabela produtos(id, nome, preco, estoque): formulário completo com as 4 operações e validações (preco > 0, estoque >= 0).

RESULTADO ESPERADO

CRUD funcional com validações e refresh; aspas escapadas em nomes.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir Escape de aspas (Replace(nome, '"', '\"')) e confirmações.

EXERCÍCIO 2 — Consulta com filtro dinâmico**PRÁTICO · LABORATÓRIO**

Text de busca + botão: SELECT ... WHERE nome LIKE '%texto%' ORDER BY nome; preencha a ListBox; mostre contagem de resultados com rs.RecordCount ou contador próprio.

RESULTADO ESPERADO

Busca parcial funcionando com LIKE montado por concatenação segura e contagem exibida.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Discuta: por que RecordCount pode vir -1 em alguns cursores? (contagem manual é portátil).

EXERCÍCIO 3 — Relatório de falhas**ESCRITO**

Do teste em dupla trocada: liste os 3 bugs mais sérios encontrados no programa avaliado, com: sintoma, causa provável e correção proposta.

RESULTADO ESPERADO

Relatório claro ligando sintoma > causa > correção (ex.: falta de validação, falta de MoveNext, aspas não escapadas).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção coletiva no projetor com os 3 melhores relatórios; vira material de revisão da A2.

TESTE RÁPIDO — MÓDULO 11: Manipulação de dados Cliente/Servidor (ADO)

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Sem `rs.MoveNext` dentro do `Do While Not rs.EOF`, o programa:

- (A) pula a primeira linha
- (B) entra em laço infinito
- (C) fecha o Recordset
- (D) dá erro de sintaxe

RESPOSTA: B

O ponteiro não avança: EOF nunca chega.

2. `rs!nota` lê:

- (A) o nome da coluna
- (B) o valor do campo nota da linha atual
- (C) a primeira linha inteira
- (D) o total de linhas

RESPOSTA: B

Acesso ao campo da linha corrente do Recordset.

3. Para escapar uma aspa simples dentro de um texto que vai virar SQL:

- (A) duplica a aspa (")
- (B) remove a aspa
- (C) troca por aspas duplas
- (D) usa \ '

RESPOSTA: A

Em SQL, " representa uma aspa literal dentro do texto.

4. cn.Execute de um UPDATE devolve:

- (A) um Recordset com as linhas alteradas
- (B) nada de tabela; afeta linhas no banco
- (C) um arquivo temporário
- (D) um MsgBox automático

RESPOSTA: B

Manipulação não retorna Recordset; consulta retorna.

5. Validar no cliente antes de enviar SQL serve para:

- (A) deixar o banco mais rápido
- (B) evitar comandos inválidos/perigosos e mensagens ruins
- (C) substituir o On Error
- (D) economizar memória

RESPOSTA: B

Defesa em camadas: cliente valida, servidor protege, erro tratado.

AVALIAÇÃO À VISTA

AVALIAÇÃO 2 (A2 — 30 pts): prova obrigatória nesta semana (aula 48) — HTML, PHP e GET/POST. Modelo completo na seção AVALIAÇÕES.

MÓDULO 12 · AULAS 65 A 72 · SEMANAS 17 A 18**Backup e recuperação de banco de dados**

PARTE V — DADOS CLIENTE/SERVIDOR E ADMINISTRAÇÃO DO BANCO

OBJETIVOS DE APRENDIZAGEM

- ✓ Explicar a finalidade de backup e os cuidados de organização das cópias.
- ✓ Executar backup do MySQL com mysqldump e restaurar com mysql <.
- ✓ Validar a recuperação restaurando em banco de teste e conferindo dados.
- ✓ Planejar rotina de cópias (frequência, nomeação, guarda) para um cenário real.

AULA 17**Backup: a cópia que salva o semestre****Por que backup (e por que agora)**

Dados são o ativo mais valioso de um sistema: discos falham, alguém executa DELETE sem WHERE, um UPDATE errado reescreve notas. **Backup** é a cópia que permite voltar no tempo; **recuperação** é o treino que garante que a cópia funciona.

Backup com mysqldump (prompt de comando)

SHELL

REM cópia estrutura + dados do banco escola:**mysqldump** -u root -paluno123 escola > c:\lp2\backups\escola_2026-09-18.sql**REM** só a estrutura (sem dados):**mysqldump** -u root -paluno123 --no-data escola > estrutura.sql**Organização das cópias**

- Nome com data: banco_YYYY-MM-DD.sql (ordena sozinho na pasta);
- Pasta própria fora do servidor (c:\lp2\backups\ ou Drive/nuvem da escola);
- Rotina mínima de laboratório: backup ao fim de cada semana de uso do banco;
- Testar a recuperação PELO MENOS uma vez por mês (cópia não testada = esperança).

Backup bom é backup restaurado

Um .sql nunca visto pode estar truncado, com senha errada ou banco errado dentro. A validação é prática: restaurar em um banco de teste e conferir contagens (SELECT COUNT(*) antes x depois).

AULA 18**Recuperação e validação pós-restauro**

Restaurar (prompt de comando)

SHELL

```

REM restaura sobre o banco escola (recria tabelas e dados do .sql):
mysql -u root -paluno123 escola < c:\lp2\backups\escola_2026-09-18.sql

REM restauração segura em banco de TESTE:
mysql -u root -paluno123 -e "CREATE DATABASE escola_teste;"
mysql -u root -paluno123 escola_teste < escola_2026-09-18.sql

```

Validação após recuperação

1. Conferir tabelas: SHOW TABLES; no banco restaurado;
2. Conferir contagens: SELECT COUNT(*) de cada tabela (antes × depois);
3. Conferir amostras: 3 linhas conhecidas batem campo a campo;
4. Registrar: data/hora do restauro, arquivo usado, responsável, resultado;
5. Só então apontar a aplicação para o banco recuperado.

Cenários de desastre (estudo de caso)

Qual cópia usar?

Cenário	Ação
DELETE sem WHERE apagou tudo às 10h	Restaurar backup de ontem + reaplicar inserções de hoje (se houver log)
Disco do servidor morreu	Restaurar último backup em servidor novo e reconfigurar conexão
UPDATE errado mudou 200 notas	Restaurar em banco de teste, extrair só a coluna correta e corrigir
Backup de sexta corrompido	Usar o de quinta (por isso guardamos várias gerações!)

DICA

Gerações de backup (avô-pai-filho): manter pelo menos as 4 últimas cópias semanais resolve 90% dos cenários acima sem drama.

EXERCÍCIOS — MÓDULO 12

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Rotina completa de backup

PRÁTICO · LABORATÓRIO

Gere backup do banco escola com nome datado; apague uma tabela de testes (DROP); restaure do .sql; valide com SHOW TABLES e COUNT(*).

RESULTADO ESPERADO

Ciclo completo executado com validação registrada no caderno.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir o nome datado do arquivo e a conferência de contagens pós-restauro.

EXERCÍCIO 2 — Recuperação cirúrgica**PRÁTICO · LABORATÓRIO**

Simule UPDATE errado em 5 notas; restaure o backup em escola_teste; escreva o SQL que copia de volta apenas as notas corretas do banco de teste para o produção (UPDATE com JOIN ou subselect, demonstrado no quadro).

RESULTADO ESPERADO

Notas corrigidas por recuperação seletiva, sem derrubar o resto dos dados.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Demonstre a versão com subselect UPDATE ... SET nota = (SELECT ... WHERE id = ...) e discuta quando ela é segura.

EXERCÍCIO 3 — Plano de backup da escola**ESCRITO**

Proponha por escrito a rotina de backup do laboratório: frequência, nomeação, local de guarda, gerações mantidas, responsável e procedimento de recuperação testada.

RESULTADO ESPERADO

Plano coerente (semanal + antes de provas; 4 gerações; Drive + pasta local; teste mensal) com justificativas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Os 3 melhores planos viram o procedimento oficial do laboratório (com crédito aos autores).

TESTE RÁPIDO — MÓDULO 12: Backup e recuperação de banco de dados

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. `mysqldump banco > arquivo.sql` produz:

- (A) um banco novo
- (B) um arquivo com estrutura + dados em SQL
- (C) um backup binário criptografado
- (D) um relatório em PDF

RESPOSTA: B

Texto SQL reproduzível em qualquer servidor MySQL.

2. Para restaurar o conteúdo do arquivo no banco:

- (A) mysqldump < arquivo
- (B) mysql banco < arquivo.sql
- (C) RESTORE FILE arquivo
- (D) cn.Execute(arquivo)

RESPOSTA: B

O cliente mysql executa o script .sql dentro do banco indicado.

3. Validar uma recuperação inclui:

- (A) só olhar o tamanho do arquivo
- (B) SHOW TABLES + contagens + amostras conferidas
- (C) reiniciar o servidor
- (D) abrir o .sql no Bloco de Notas

RESPOSTA: B

Conferência estrutural e de conteúdo após o restauro.

4. Manter várias gerações de backup (avô-pai-filho) protege contra:

- (A) falta de espaço
- (B) cópias corrompidas ou desastres descobertos tardiamente
- (C) senhas fracas
- (D) vírus de tela

RESPOSTA: B

Se a cópia de ontem está ruim, a de quinta-feira salva.

5. Restaurar primeiro em banco de TESTE serve para:

- (A) treinar SQL
- (B) não arriscar o banco de produção durante a validação
- (C) acelerar o mysqldump
- (D) economizar disco

RESPOSTA: B

Valida o arquivo e o procedimento sem tocar em dados reais.

PARTE VI

SAÍDAS, DISTRIBUIÇÃO, GRÁFICOS E PROJETO INTEGRADOR

AULAS 73 A 96 · SEMANAS 19 A 24

Módulo 13 — Relatórios, impressão e documentos fiscais

Módulo 14 — Disco de instalação e recursos gráficos

Módulo 15 — Projeto integrador e consolidação final

A Parte VI reúne saídas (relatórios/impressão), distribuição (disco de instalação), gráficos e o projeto integrador das semanas 23–24.

MÓDULO 13 · AULAS 73 A 80 · SEMANAS 19 A 20

Relatórios, impressão e documentos fiscais

PARTE VI — SAÍDAS, DISTRIBUIÇÃO, GRÁFICOS E PROJETO INTEGRADOR

OBJETIVOS DE APRENDIZAGEM

- ✓ Gerar relatórios com seleção e organização de dados (tela e papel).
- ✓ Usar o objeto Printer (Print, Tab, String, EndDoc) para saídas impressas.
- ✓ Montar documentos fiscais simples (cupom/nota) com layout controlado.
- ✓ Aplicar Format e alinhamentos para saída profissional.

AULA 19

Relatórios: dados que viram informação

Anatomia de um relatório

- **Cabeçalho:** título, empresa, data/hora de emissão, página;
- **Colunas:** nomes alinhados sobre os dados;
- **Corpo:** linhas selecionadas e ordenadas (SQL faz o pesado);
- **Rodapé de grupo/geral:** contagens, somas, médias;
- **Separadores:** linhas String(80, "-") entre blocos.

Relatório de aprovados na tela (montado por Sub)

VB

```
Public Sub RelatorioAprovados()  
Dim rs As ADODB.Recordset  
Print Tab(20); "RELATÓRIO DE APROVADOS - " & Format(Date, "dd/mm/yyyy")  
Print String(60, "-")  
Print "NOME"; Tab(35); "NOTA"  
Print String(60, "-")  
Set rs = cn.Execute("SELECT nome, nota FROM alunos " & _  
"WHERE nota >= 7 ORDER BY nome")  
Do While Not rs.EOF  
Print rs!nome; Tab(35); Format(rs!nota, "0.0")  
rs.MoveNext  
Loop  
rs.Close  
Print String(60, "-")  
End Sub
```

Do tela para o papel: objeto Printer

Impressão com Printer

VB

```
Public Sub ImprimirRelatorio()
Printer.Print Tab(20); "RELATÓRIO DE APROVADOS"
Printer.Print String(60, "-")
' ...mesmas linhas do corpo, trocando Print por Printer.Print...
Printer.Print String(60, "-")
Printer.Print "Emitido em " & Now
Printer.EndDoc ' envia o trabalho à impressora'
End Sub
```

Printer = a impressora como objeto

Tudo que você Printa no Printer entra na fila de impressão; **EndDoc** fecha o documento e despacha. Sem EndDoc, o trabalho fica preso. Tab() e String() cuidam do alinhamento em colunas de caracteres.

AULA 20**Documentos fiscais: cupom e nota simplificados****Layout de cupom (monoespaçado)**

Cupom de venda não fiscal (modelo didático)

VB

```
Public Sub Cupom(venda As Integer)
Printer.Print Tab(12); "MERCADO ESCOLA LTDA"
Printer.Print Tab(8); "CNPJ 00.000.000/0001-00"
Printer.Print String(48, "-")
Printer.Print "CUPOM NÃO FISCAL - Venda "; venda
Printer.Print String(48, "-")
' itens: codigo, descricao, qtd, unit, total
Printer.Print "001 ARROZ 5KG 2 x 22,90 45,80"
Printer.Print String(48, "-")
Printer.Print Tab(30); "TOTAL R$ 45,80"
Printer.Print "Obrigado pela preferência!"
Printer.EndDoc
End Sub
```

O que um documento fiscal de verdade exige (contexto)

- Identificação do emitente (razão social, CNPJ, endereço);
- Numeração sequencial e data/hora de emissão;
- Itens com quantidade, unitário e total conferindo (soma validada);
- Totais e forma de pagamento;
- Em sistemas reais: autorização da SEFAZ (NF-e/NFC-e) — aqui, o MODELO didático.

DICA

Monte os itens do cupom vindo do banco (tabela itens_venda) com Recordset: o cupom deixa de ser texto fixo e vira saída de dados — exatamente como no projeto final.

ATENÇÃO

Alinhamento em impressora real muda com fonte proporcional: fixe fonte monoespaçada (Printer.FontName = "Courier New") para as colunas fecharem.

EXERCÍCIOS — MÓDULO 13

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Relatório de estoque**PRÁTICO · LABORATÓRIO**

Relatório (tela + Printer) dos produtos com estoque < mínimo: cabeçalho com data, colunas código/descrição/estoque, rodapé com contagem e valor total imobilizado.

RESULTADO ESPERADO

Saída alinhada com Tab/String e totais corretos; impressão (ou PDF) conferida.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir o SQL com WHERE e o acumulador do valor imobilizado.

EXERCÍCIO 2 — Cupom dinâmico**PRÁTICO · LABORATÓRIO**

Cupom que lê a venda pelo id: cabeçalho do emitente (tabela loja), itens da tabela itens_venda via Recordset, totais calculados (qtd × unit) e EndDoc.

RESULTADO ESPERADO

Cupom impresso/PDF com itens reais do banco e total batendo com a soma manual.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Validação cruzada: soma manual dos itens × total impresso.

EXERCÍCIO 3 — Leitura de documento fiscal**ESCRITO**

Traga (ou receba) um cupom/NF-e real e identifique por escrito: emitente, numeração, data, itens com unit/qtd/total, totais e pagamento; marque o que nosso modelo didático já reproduz e o que ficaria por conta da SEFAZ.

RESULTADO ESPERADO

Identificação completa dos blocos e separação clara didático × fiscal real.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Discussão de 10 min: por que a autorização fiscal existe (contexto profissional).

TESTE RÁPIDO — MÓDULO 13: Relatórios, impressão e documentos fiscais

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Para alinhar colunas em relatórios de texto usamos:

- (A) Space() apenas
- (B) Tab(n) e String(n, "-")
- (C) MsgBox
- (D) Printer.Page

RESPOSTA: B

Tab posiciona colunas; String desenha separadores.

2. Sem Printer.EndDoc, o trabalho de impressão:

- (A) sai imediatamente
- (B) fica retido/no buffer sem despachar
- (C) é cancelado
- (D) imprime em dobro

RESPOSTA: B

EndDoc fecha o documento e envia à fila.

3. Para colunas fecharem em impressora real, a fonte deve ser:

- (A) proporcional (Arial)
- (B) monoespçada (Courier New)
- (C) itálica
- (D) decorativa

RESPOSTA: B

Largura fixa por caractere = colunas alinhadas.

4. Os itens de um cupom 'de verdade' no nosso sistema vêm:

- (A) digitados fixos no código
- (B) do banco, via Recordset
- (C) de um arquivo .bmp
- (D) do Clipboard

RESPOSTA: B

Saída de dados: SQL + Recordset alimentam o layout.

5. Um relatório gerencial típico termina com:

- (A) somente os detalhes
- (B) totais/contagens/médias de fechamento
- (C) o código SQL
- (D) a string de conexão

RESPOSTA: B

Rodapé de fechamento dá a leitura gerencial do conjunto.

MÓDULO 14 · AULAS 81 A 88 · SEMANAS 21 A 22**Disco de instalação e recursos gráficos**

PARTE VI — SAÍDAS, DISTRIBUIÇÃO, GRÁFICOS E PROJETO INTEGRADOR

OBJETIVOS DE APRENDIZAGEM

- ✓ Preparar e organizar os arquivos necessários à distribuição do programa.
- ✓ Gerar o pacote/disco de instalação com o Package & Deployment Wizard.
- ✓ Aplicar recursos gráficos no VB6 (PictureBox: Line, Circle, PSet) com herança do modo DOS.
- ✓ Testar a instalação em máquina/conta limpa.

AULA 21**Disco de instalação: do projeto ao pacote****O que vai dentro de um instalador**

- Executável compilado (.exe) do projeto;
- Bibliotecas de tempo de execução (runtime VB6, MSADO/ODBC);
- Arquivos de dados/modelos (textos, logs, templates de relatório);
- Configurações (string de conexão parametrizável!);
- Atalhos e desinstalador.

Package & Deployment Wizard passo a passo

1. No VB6: Add-Ins > Package & Deployment Wizard;
2. Compile o projeto primeiro (File > Make projeto.exe);
3. Escolha o projeto e a opção **Package**;
4. Selecione os arquivos dependentes sugeridos (marque os .bas de dados e templates);
5. Defina o grupo de menu/atalhos e o destino padrão (C:\MeuSistema);
6. Gere a pasta/pacote (setup.exe + cab) e copie para a pasta de distribuição;
7. Teste em conta/máquina limpa: instalar, executar, conectar ao banco, desinstalar.

Configuração fora do executável

Senha/servidor nunca vão compilados no .exe: o instalador copia um **config.txt/ini** que o programa lê ao abrir (conexão parametrizável). Assim o mesmo pacote serve para laboratórios diferentes.

ATENÇÃO

Teste de instalação que não passa em máquina limpa = pacote incompleto (falta dependência). O erro clássico é esquecer o driver ODBC na lista de pré-requisitos documentados.

AULA 22**Recursos gráficos: a herança do modo DOS**

Do DOS ao formulário

A ementa cita 'programação gráfica em ambiente DOS': nos PCs antigos, gráficos nasciam de comandos de desenho em modo texto/gráfico (SCREEN, LINE, CIRCLE). No VB6, o mesmo espírito vive no **PictureBox**: coordenadas, cores e métodos de desenho.

Desenhando no PictureBox

VB

```
Picture1.ScaleMode = 3 ' pixels
Picture1.BackColor = vbWhite

Picture1.Line (10, 10)-(200, 120), vbBlue, B ' retângulo (B = box)
Picture1.Circle (110, 65), 40, vbRed ' círculo
Picture1.Line (10, 130)-(200, 130), vbBlack ' linha simples

Dim x As Integer
For x = 10 To 200 Step 5
    Picture1.PSet (x, 140 + 10 * Sin(x / 10)), vbGreen ' pontos (onda)
Next x
```

Gráficos com dados do sistema

Gráfico de barras das notas (dados do banco)

VB

```
Dim rs As ADODB.Recordset, x As Integer
Set rs = cn.Execute("SELECT nome, nota FROM alunos ORDER BY nome")
x = 20
Do While Not rs.EOF
    Picture1.Line (x, 160)-(x + 18, 160 - rs!nota * 14), vbBlue, BF
    Picture1.CurrentX = x: Picture1.CurrentY = 165
    Picture1.Print Left$(rs!nome, 3)
    x = x + 26
    rs.MoveNext
Loop
rs.Close
```

DICA

Gráfico de barras com BF (box filled) + escala simples (nota × 14) transforma qualquer consulta em painel visual — use no dashboard do projeto final.

EXERCÍCIOS — MÓDULO 14

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Pacote de instalação

PRÁTICO · LABORATÓRIO

Compile um projeto seu (ex.: CRUD de produtos) e gere o pacote com o Wizard; teste em conta limpa: instalar, rodar, conectar, desinstalar. Registre o passo a passo com prints.

RESULTADO ESPERADO

Pacote funcional com teste em ambiente limpo documentado.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conferir presença do config.ini parametrizável e do driver ODBC na documentação do pacote.

EXERCÍCIO 2 — Bandeira e onda**PRÁTICO · LABORATÓRIO**

No PictureBox: desenhe a bandeira da escola (retângulos/círculos com B/BF) e uma onda com PSet em laço; adicione botão Limpar (Picture1.Cls).

RESULTADO ESPERADO

Duas composições gráficas com laços e botão de limpeza.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Valorize o uso de laços (não 40 linhas de PSet à mão).

EXERCÍCIO 3 — Dashboard de notas**PRÁTICO · LABORATÓRIO**

Gráfico de barras das notas por aluno (consulta SQL) com eixo, rótulos (Left\$ do nome) e legenda; inclua o dashboard como tela do seu projeto.

RESULTADO ESPERADO

Barras proporcionais às notas com rótulos legíveis.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Integração direta com o projeto integrador (semana 23): já deixe a tela pronta.

TESTE RÁPIDO — MÓDULO 14: Disco de instalação e recursos gráficos

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. O Package & Deployment Wizard gera:

- (A) o banco de dados
- (B) o pacote/disco de instalação (setup + dependentes)
- (C) o relatório fiscal
- (D) o backup do MySQL

RESPOSTA: B

Empacota exe, runtimes, dados e atalhos para distribuição.

2. Senha/servidor de conexão devem ficar:

- (A) compilados no .exe
- (B) em arquivo de configuração lido em tempo de execução
- (C) no título do formulário
- (D) no Registry sem documentação

RESPOSTA: B

Config.ini parametrizável: mesmo pacote em ambientes diferentes.

3. No PictureBox, o método que desenha um retângulo preenchido usa:

- (A) Box F
- (B) BF no final do Line
- (C) Fill automaticamente
- (D) Rect()

RESPOSTA: B

Line (x1,y1)-(x2,y2), cor, BF (B = borda, BF = preenchido).

4. PSet serve para:

- (A) desenhar um ponto na coordenada
- (B) imprimir texto
- (C) limpar a tela
- (D) mover o formulário

RESPOSTA: A

Ponto único — ondas e texturas nascem de laços com PSet.

5. Testar a instalação em máquina/conta limpa serve para:

- (A) ganhar tempo
- (B) garantir que o pacote leva todas as dependências
- (C) economizar licença
- (D) validar o banco

RESPOSTA: B

Ambiente limpo revela dependência esquecida no pacote.

MÓDULO 15 · AULAS 89 A 96 · SEMANAS 23 A 24**Projeto integrador e consolidação final**

PARTE VI — SAÍDAS, DISTRIBUIÇÃO, GRÁFICOS E PROJETO INTEGRADOR

OBJETIVOS DE APRENDIZAGEM

- ✓ Integrar banco + SQL + estruturas + modularização + arquivos + relatórios + gráficos no projeto.
- ✓ Aplicar revisão geral da ementa como preparação da prova final.
- ✓ Testar, revisar e apresentar o projeto com demonstração completa.
- ✓ Consolidar o portfólio do semestre: código, pacote de instalação e documentação.

AULA 23**Integração do projeto (semana 23)****O que o projeto precisa ter**

- **Banco:** tabelas criadas por script .sql versionado;
- **Conexão:** módulo Conexao.bas com tratamento de falhas;
- **CRUD:** cadastro e consulta via ADO com validação no cliente;
- **Estruturas:** vetor/matriz/registro onde fizer sentido (ex.: cache, cálculos);
- **Modularização:** regras em Modules (Functions puras);
- **Arquivos:** importação/exportação de dados (txt) com validação;
- **Relatório:** saída Printer/PDF com totais; **gráfico** PictureBox opcional;
- **Instalação:** pacote gerado + config.ini parametrizável.

Roteiro de acompanhamento por aula

Semana 23 — aulas 89 a 92

Aula	Foco	Entrega
89	Integração: banco + SQL + telas (CRUD completo)	Entrega 2
90	Arquivos + relatório acoplados ao fluxo	—
91	Modularização final + gráficos/dashboard	—
92	Testes internos com plano de testes da dupla	Entrega 3

DICA

Congele escopo na semana 23: o que não entrou até a aula 92 vira 'versão 2' no relatório, não dívida na apresentação.

AULA 24**Consolidação, prova final e apresentações (semana 24)****Revisão geral (aulas 93–94)**

- SGBD/conexão: string, On Error, abrir/fechar;
- SQL: CREATE/INSERT/UPDATE/DELETE/SELECT com WHERE/ORDER/GROUP;
- Estruturas: vetor, matriz, registro; Strings e funções;
- Modularização: Sub/Function, ByVal/ByRef, escopos;
- Arquivos: Output/Append/Input, EOF, FreeFile, Split;
- Cliente/Servidor: Recordset (EOF/MoveNext/rs!campo);
- Backup/recuperação: mysqldump/mysql <, validação;
- Relatórios/Printer e gráficos PictureBox; pacote de instalação.

Prova final escrita (30 pts) e apresentações (10 pts)

1. Aulas 93–94: revisão ativa com o modelo de prova da apostila (em duplas, correção comentada);
2. Prova escrita individual e cumulativa conforme o Plano de Ensino;
3. Aula 95: ensaio geral cronometrado (5–8 min) com demonstração: cadastro válido, tentativa inválida, consulta/relatório ao vivo;
4. Aula 96: apresentações + entrega do pacote de instalação + encerramento.

Demonstração que convence

Mostre o fluxo completo com dados reais: cadastre > erre de propósito (validação) > corrija > consulte > imprima relatório. Erro tratado ao vivo vale mais que demo perfeita ensaiada demais.

EXERCÍCIOS — MÓDULO 15

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Checklist de integração**EM GRUPO**

Preencham o checklist de integração do projeto (8 itens da aula 89) com status Feito/Parcial/Pendente e plano para os pendentes até a aula 92.

RESULTADO ESPERADO

Checklist realista com plano de ação datado.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Visto do professor em cada dupla; pendências viram acompanhamento da semana 24.

EXERCÍCIO 2 — Plano de testes do projeto**EM GRUPO**

Montem a tabela de testes com mínimo 8 casos: cadastro válido/inválido, consulta com e sem resultado, exclusão confirmada/cancelada, importação com linha corrompida, relatório com zero registros, conexão com serviço parado.

RESULTADO ESPERADO

Tabela executada com resultados OK/FALHOU e correções aplicadas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Troca de testes entre duplas na aula 92 (olho novo acha bug novo).

EXERCÍCIO 3 — Ensaio cronometrado**EM GRUPO**

Ensaio de 5–8 min com cronômetro: abertura (tema/tabela), demonstração completa (válido, inválido, consulta, relatório), fechamento (o que ficou para a versão 2). Gravem no celular como plano B.

RESULTADO ESPERADO

Apresentação dentro do tempo com demonstração de erro tratado e vídeo de backup.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Feedback sanduíche em 2 min por dupla: ponto forte, ajuste, ponto forte.

TESTE RÁPIDO — MÓDULO 15: Projeto integrador e consolidação final

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. O checklist de integração do projeto exige, entre outros:

- (A) somente telas bonitas
- (B) banco + CRUD + módulos + arquivos + relatório + pacote
- (C) apenas o relatório
- (D) somente backup

RESPOSTA: B

Integração é o nome do jogo: todas as camadas do semestre.

2. Congelar escopo na semana 23 significa:

- (A) não testar mais
- (B) não acrescentar funcionalidades novas; pendências viram 'versão 2'
- (C) parar de programar
- (D) adiar a apresentação

RESPOSTA: B

Escopo fechado = entrega e apresentação confiáveis.

3. Na demonstração, mostrar uma entrada inválida sendo barrada prova:

- (A) que o programa tem bugs
- (B) que a validação funciona
- (C) que o banco está lento
- (D) que faltou teste

RESPOSTA: B

Erro tratado ao vivo é demonstração de qualidade, não defeito.

4. A prova final (30 pts) é:

- (A) em duplas com consulta
- (B) escrita, individual e cumulativa
- (C) oral
- (D) só do projeto

RESPOSTA: B

Conforme o Plano de Ensino; os 10 pts restantes são do projeto.

5. O pacote de instalação entregue deve incluir:

- (A) somente o .exe
- (B) exe + dependentes + config.ini parametrizável + atalhos
- (C) o banco com senhas compiladas
- (D) os fontes sem compilar

RESPOSTA: B

Distribuição profissional: instalável e configurável por ambiente.

AVALIAÇÃO À VISTA

AVALIAÇÃO 3 (A3 — 40 pts): prova escrita final individual (30 pts, aulas 70–71) + projeto e apresentação (10 pts, aula 72). Modelo e rubrica na seção AVALIAÇÕES.

AULAS 49 A 72 · SEMANAS 17 A 24 · 6 ENTREGAS**Projeto Final Integrador**

O que é: Programa completo em Visual Basic 6.0 que reúne, conforme a viabilidade do ambiente: conexão com banco de dados (MySQL via ODBC/ADO), instruções SQL de manipulação e consulta, estruturas de dados (vetor, matriz e/ou registro), modularização (Sub e Function em Modules), variáveis String, arquivos/textos para importação/exportação, geração de relatório e/ou saída para impressão, recursos gráficos quando pertinente e preparação do disco/pacote de instalação.

Temas sugeridos no material didático: Controle de estoque com relatórios · Cadastro de alunos com histórico · Contas a pagar/receber com documentos · Biblioteca com empréstimos · PDV simples com cupom (documento fiscal) · Controle de tarefas com arquivos.

Entregas do projeto

Entrega	Descrição	Aulas	Semana
Entrega 1	Requisitos e desenho: tabelas do banco, telas e módulos previstos	Aulas 89–90	Semana 23
Entrega 2	Versão parcial: conexão + SQL (cadastro e consulta) + estruturas	Aulas 91–92	Semana 23
Entrega 3	Versão completa: arquivos, relatório/impressão e modularização	Aulas 93–94	Semana 24
Entrega 4	Pacote de instalação + apresentação e defesa do projeto	Aulas 95–96	Semana 24

Critérios sugeridos para os 10 pts (S)

Critério	Pontos	O que se espera
Funcionalidade	3,0	Cadastro, consulta e relatório funcionando de ponta a ponta com o banco.
Estruturas e modularização	2,0	Uso coerente de vetor/matriz/registro e Sub/Function em Modules.
SQL e banco de dados	2,0	CREATE/INSERT/SELECT/UPDATE corretos; conexão tratada com falhas.
Relatório e saída	1,0	Relatório organizado (cabeçalho, linhas, totais) e/ou documento impresso.
Instalação e apresentação	2,0	Pacote de instalação gerado + apresentação clara com demonstração.

DICA

O desenvolvimento do projeto é guiado aula a aula no Módulo 12 desta apostila: requisitos (aula 50), estrutura (51), páginas (52–55), testes (58), correção de erros (59) e apresentação (70–72).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Acompanhe as 6 entregas com vistos e feedback curto. A nota de projeto (10 pts) considera o processo (entregas 1–5) e a apresentação final. Sugestão: teste cruzado entre grupos na aula 69 e ensaio geral na 71.

A1 · 30 PTS & A2 · 30 PTS & A3 · 40 PTS**Avaliações — 100 pontos no semestre**

Regras oficiais do Plano de Ensino e proposta de operacionalização do cronograma (S). Os modelos de prova abaixo são **propostas (S)** — o professor pode adaptá-los.

Regras oficiais

Avaliação	Pontos	Regra oficial
1ª Avaliação	30 pts	Distribuídos a critério do professor (trabalhos, seminários, testes etc.), admitindo-se inclusive provas.
2ª Avaliação	30 pts	Obrigatoriamente por meio de prova; poderá ser em duplas ou com consultas.
3ª Avaliação	40 pts	30 pts obrigatoriamente em prova escrita final individual + 10 pts distribuídos a critério do professor.

Proposta de operacionalização no cronograma (S)

Etapa	Período	Proposta
1ª Avaliação	Semanas 1 a 8	Lista prática de SQL + estruturas (10 pts — entrega semana 6) + atividades AVA (5 pts) + teste escrito-prático de SGBD/SQL/estruturas (15 pts — semana 8). (S)
2ª Avaliação	Semana 16	Prova obrigatória (pode ser em duplas ou com consulta — Plano de Ensino): Strings, modularização, arquivos e Cliente/Servidor.
3ª Avaliação	Semana 24	Prova escrita final individual e cumulativa (30 pts) + projeto integrador e apresentação (10 pts — critério do professor).

APOIO E RECUPERAÇÃO (PROPOSTA — S)

Identificação precoce de dificuldades pelos exercícios de cada aula; revisões estratégicas ao final dos blocos (aulas 36 e 60); listas de reforço e atividades diferenciadas no AVA; reforço dos pré-requisitos de HTML antes do PHP e de lógica antes de formulários; nova oportunidade de demonstrar aprendizagem por meio das entregas do projeto final.

AVALIAÇÃO 1 (A1) — 30 PONTOS · SEMANAS 1 A 8 (S)

Trabalho HTML (10) + atividades AVA (5) + teste escrito-prático (15)

Componente	Pontos	Quando
Lista prática de SQL + estruturas de dados	10 pts	Entrega na semana 6

Componente	Pontos	Quando
Atividades extraclasse/AVA e exercícios	5 pts	Semanas 1 a 8
Teste escrito-prático (SGBD, conexão, SQL, estruturas)	15 pts	Semana 8

Lista prática (10 pts): SQL + estruturas no laboratório

Resolver no laboratório: (a) criação de banco e tabela no MySQL; (b) 5 INSERT, 3 UPDATE e 2 DELETE; (c) 4 SELECT com WHERE e ORDER BY; (d) programa VB6 com um vetor de 10 números (média, maior, menor) e uma matriz 3×3 somada por linha. Individual ou em duplas (critério do professor).

Requisitos do trabalho:

- Script .sql completo (CREATE DATABASE, CREATE TABLE, INSERT/UPDATE/DELETE/SELECT) comentado;
- Capturas de tela das execuções no MySQL (ou saída colada no documento);
- Projeto VB6 com vetor e matriz processados (código + tela de saída);
- Organização: pasta unica nomeada sobrenome_lista1 com .sql, .vbp e documento de respostas.

Rubrica de correção do trabalho (10 pts)

Critério	Pontos	O que se espera
Criação e população do banco	2,5	CREATE/INSERT corretos, tipos adequados, chave primária.
Manipulação (UPDATE/DELETE)	2,5	Comandos corretos com WHERE seguro (sem apagar tudo sem querer).
Consultas (SELECT)	2,5	Filtros e ordenação corretos; resultados conferem.
Vetor e matriz em VB6	2,5	Declaração, laços e cálculos corretos; saída legível.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Aplique a rubrica item a item com visto no checklist; devolva com feedback escrito. Trabalhos com links quebrados perdem o critério 'Estrutura e navegação' integralmente. Combine a entrega via AVA/Drive + pasta do laboratório.

Teste escrito-prático (15 pts) — modelo (S)

AVALIAÇÃO 1 — TESTE ESCRITO-PRÁTICO (15 pts) · WEB E HTML

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Prova individual, sem consulta. Duração: 2 aulas (100 min).

- Parte A: 6 questões objetivas (1 pt cada). Parte B: 3 questões práticas (3 pts cada).
- Ambiente de referência: Visual Basic 6.0 + MySQL via ODBC/ADO.

Parte A — questões objetivas (1 pt cada)

Questão 1. Um SGBD (Sistema Gerenciador de Banco de Dados) é:

- (A) Um editor de textos para dados.
- (B) Um software que armazena, organiza e controla o acesso aos dados (ex.: MySQL).
- (C) Uma linguagem de programação estruturada.
- (D) Um periférico de armazenamento.

Resposta: B

SGBD gerencia criação, consulta, alteração e segurança dos dados.

Questão 2. A string de conexão em ADO serve para:

- (A) Definir a cor do formulário.
- (B) Informar driver, servidor, banco e credenciais para abrir a conexão.
- (C) Criar as tabelas do banco automaticamente.
- (D) Compilar o programa VB6.

Resposta: B

Driver/Servidor/Database/Uid/Pwd: tudo que a conexão precisa.

Questão 3. Para INSERIR uma linha nova em uma tabela usamos:

- (A) SELECT
- (B) UPDATE
- (C) INSERT
- (D) ALTER

Resposta: C

INSERT INTO tabela (cols) VALUES (...). UPDATE altera; SELECT consulta.

Questão 4. A cláusula que FILTRA linhas de um SELECT é:

- (A) ORDER BY
- (B) WHERE
- (C) GROUP BY
- (D) VALUES

Resposta: B

WHERE define a condição; ORDER BY ordena o resultado.

Questão 5. Em VB6, Dim v(1 To 5) As Integer declara:

- (A) Uma matriz 5×5
- (B) Um vetor com 5 posições (1 a 5)
- (C) Um registro com 5 campos
- (D) 5 variáveis soltas

Resposta: B

Vetor (array unidimensional) com índices de 1 a 5.

Questão 6. Após usar um arquivo aberto com Open ... For Output, é obrigatório:

- (A) Delete #1
- (B) Close #1
- (C) Print #1
- (D) Reset #1

Resposta: B

Close libera o arquivo e garante a gravação completa.

Parte B — questões práticas (3 pts cada)

Questão 7. Escreva o script SQL que: cria a tabela alunos (id inteiro, chave primária auto-incremento; nome VARCHAR(40) não nulo; nota DECIMAL(3,1)); insere 2 alunos; e consulta nome e nota dos aprovados (nota >= 7) em ordem alfabética.

GABARITO — código modelo

```
CREATE TABLE alunos (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(40) NOT NULL,  
  nota DECIMAL(3,1)  
);  
INSERT INTO alunos (nome, nota) VALUES ('Ana Silva', 8.5);  
INSERT INTO alunos (nome, nota) VALUES ('Bruno Souza', 6.8);  
SELECT nome, nota FROM alunos  
WHERE nota >= 7  
ORDER BY nome;
```

Crêterios: CREATE com PK/auto-incremento e tipos (1,0) · 2 INSERT corretos (1,0) · SELECT com WHERE e ORDER BY (1,0).

Questão 8. O código VB6 abaixo deveria abrir a conexão com o MySQL, mas tem 3 erros. Reescreva-o corrigido.

Código para análise

PHP

```
Public cn As ADODB.Connection  
  
Public Sub AbrirConexao()  
  cn = New ADODB.Connection  
  cn.ConnectionString = "Server=localhost;Database=escola;Uid=root;Pwd=1234;"  
End Sub
```

GABARITO — código modelo

```
Public cn As ADODB.Connection  
Public Sub AbrirConexao()  
  Set cn = New ADODB.Connection  
  cn.ConnectionString = "Driver={MySQL ODBC 8.0 Driver};" & _  
    "Server=localhost;Database=escola;" & _  
    "Uid=root;Pwd=1234;"  
  cn.Open  
End Sub
```

Crêterios: 1 pt por erro: (1) faltou Set; (2) faltou o Driver ODBC na string; (3) faltou cn.Open.

Questão 9. Escreva uma Sub em VB6 que leia um vetor de 5 números inteiros digitados (InputBox), calcule a média e adicione ao List1 apenas os valores acima da média, usando For...Next.

GABARITO — código modelo

```
Public Sub AcimaDaMedia()  
    Dim v(1 To 5) As Integer  
    Dim i As Integer, soma As Single, media As Single  
    For i = 1 To 5  
        v(i) = Val(InputBox("Número " & i))  
        soma = soma + v(i)  
    Next i  
    media = soma / 5  
    For i = 1 To 5  
        If v(i) > media Then  
            List1.AddItem v(i)  
        End If  
    Next i  
End Sub
```

Critérios: Declaração e leitura com laço (1,0) · média correta (1,0) · filtro com If e AddItems (1,0).

AVALIAÇÃO 2 (A2) — 30 PONTOS · SEMANA 16, AULA 48 (S)

Prova obrigatória (pode ser em duplas ou com consulta) — HTML + PHP + GET/POST

AVALIAÇÃO 2 — PROVA (30 pts) · HTML, PHP E FORMULÁRIOS

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Conteúdo: Strings, modularização (Sub/Function), arquivos/textos, ADO/ODBC e Cliente/Servidor.
- Modalidade definida pelo professor conforme o Plano de Ensino (duplas e/ou consulta).
- Questões 1–4: conceituais. Questões 5–10: práticas (prever saída, corrigir e escrever código).
- Duração sugerida: 2 aulas (100 min).

Questão 1. Em VB6, uma Function diferencia-se de uma Sub porque:

- (A) não recebe parâmetros.
- (B) retorna um valor ao chamador.
- (C) só pode ficar em formulários.
- (D) executa mais rápido.

Resposta: B

Function devolve valor (retorno); Sub executa ações sem retornar.

Questão 2. Para ACRESCENTAR linhas ao final de um arquivo texto existente, abre-se com:

- (A) For Output
- (B) For Input

- (C) For Append
- (D) For Random

Resposta: C

Append preserva o conteúdo e grava no fim; Output recria o arquivo vazio.

Questão 3. No modelo Cliente/Servidor com banco de dados:

- (A) o banco roda no formulário do VB6.
- (B) a aplicação (cliente) envia comandos SQL e o SGBD (servidor) processa e responde.
- (C) o servidor não valida nada.
- (D) ODBC é o nome do banco de dados.

Resposta: B

Cliente requisita (ADO/ODBC), servidor SGBD executa e devolve resultados.

Questão 4. Mid\$("LINGUAGEM", 3, 4) retorna:

- (A) "LING"
- (B) "NGUA"
- (C) "GUAG"
- (D) "AGEM"

Resposta: B

Mid\$(texto, início, quantidade): posições 3 a 6 = N, G, U, A.

Questão 5. Qual é a saída exibida no List1?**Código para análise**

PHP

```
Dim s As String
s = " prova final "
List1.AddItem UCase$(Trim$(s))
List1.AddItem Len(s)
List1.AddItem Len(Trim$(s))
```

GABARITO

PROVA FINAL · 15 · 11 (Trim remove os 4 espaços: 2 antes + 2 depois).

Crêterios: 1,5 pt pela linha em maiúsculas sem espaços; 1,5 pt pelos dois comprimentos corretos.

Questão 6. O que o código exibe?

Código para análise

PHP

```
Public Type Aluno
nome As String * 20
nota As Single
End Type

Dim a(1 To 2) As Aluno
a(1).nome = "Ana": a(1).nota = 8
a(2).nome = "Beto": a(2).nota = 5

Dim i As Integer
For i = 1 To 2
If a(i).nota >= 6 Then Debug.Print a(i).nome; " aprovado"
Next i
```

GABARITO

Somente: 'Ana aprovado' — registro (Type) em vetor, com campo acessado por ponto.

Critérios: Saída correta (1,5) + explicação de Type/vetor de registros (1,5).

Questão 7. O código deveria gravar 3 nomes em alunos.txt, mas tem 2 erros. Corrija.

Código para análise

PHP

```
Open "c:\dados\alunos.txt" For Input As #1
Print #1, "Ana"
Print #1, "Beto"
Print #1, "Carla"
```

GABARITO — código modelo

```
Open "c:\dados\alunos.txt" For Output As #1
Print #1, "Ana"
Print #1, "Beto"
Print #1, "Carla"
Close #1
```

Critérios: 1,5 pt: For Output (Input só lê). 1,5 pt: faltou Close #1.

Questão 8. Escreva uma Function Media(n1 As Single, n2 As Single) As Single e uma Sub que a use: leia duas notas por InputBox e mostre MsgBox com a média e a situação (Aprovado >= 7, senão Recuperação).

GABARITO — código modelo

```
Public Function Media(n1 As Single, n2 As Single) As Single
Media = (n1 + n2) / 2
End Function
Public Sub Situacao()
Dim m As Single
m = Media(Val(InputBox("Nota 1")), Val(InputBox("Nota 2")))
If m >= 7 Then
MsgBox "Média " & m & " - Aprovado"
Else
MsgBox "Média " & m & " - Recuperação"
End If
End Sub
```

Critérios: Function com retorno (1,0) · chamada com Val/InputBox (1,0) · If/MsgBox corretos (1,0).

Questão 9. Escreva o código VB6/ADO que consulte todos os alunos com nota >= 7 e adicione nome e nota ao List1, percorrendo o Recordset corretamente.

GABARITO — código modelo

```
Dim rs As ADODB.Recordset
Set rs = cn.Execute("SELECT nome, nota FROM alunos WHERE nota >= 7")
Do While Not rs.EOF
List1.AddItem rs!nome & " - " & rs!nota
rs.MoveNext
Loop
rs.Close
```

Critérios: Execute com SELECT (1,0) · laço Do While Not rs.EOF (1,0) · MoveNext + Close (1,0).

Questão 10. Escreva uma Sub que LEIA o arquivo alunos.txt (uma linha por nome) até o fim e adicione cada linha ao List1.

GABARITO — código modelo

```
Public Sub CarregarLista()
Dim linha As String
Open "c:\dados\alunos.txt" For Input As #1
Do While Not EOF(1)
Line Input #1, linha
List1.AddItem linha
Loop
Close #1
End Sub
```

Critérios: For Input + EOF(1) (1,0) · Line Input e AddItem (1,0) · Close (1,0).

AVALIAÇÃO 3 (A3) — 40 PONTOS · SEMANA 24

Prova escrita final individual (30 pts, aulas 70–71 — S) + projeto e apresentação (10 pts, aula 72)

AVALIAÇÃO 3 — PROVA ESCRITA FINAL INDIVIDUAL (30 pts) · CUMULATIVA

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Prova individual, sem consulta, cobrindo TODA a ementa (SGBD > gráficos/projeto).
- Duração: 2 a 3 aulas (a definir pelo professor).
- Questões 1–5: objetivas/conceituais. Questões 6–10: práticas.

Questão 1. Backup e recuperação de banco de dados servem, respectivamente, para:

- (A) acelerar consultas e indexar tabelas.
- (B) gerar cópia de segurança dos dados e restaurá-los após perda/falha.
- (C) criptografar senhas e criar usuários.
- (D) imprimir relatórios e excluir tabelas.

Resposta: B

Backup = cópia (ex.: mysqldump); recuperação = restaurar a cópia e validar.

Questão 2. No VB6, o objeto usado para enviar saída formatada à impressora é:

- (A) PictureBox
- (B) Printer
- (C) ListBox
- (D) Timer

Resposta: B

Printer.Print / Printer.EndDoc; PictureBox é para recursos gráficos em tela.

Questão 3. O Package & Deployment Wizard do VB6 é usado para:

- (A) depurar erros de sintaxe.
- (B) gerar o pacote/disco de instalação do programa.
- (C) criar o banco de dados.
- (D) desenhar formulários.

Resposta: B

Ele empacota executável, DLLs e dependências para distribuição/instalação.

Questão 4. Uma matriz Dim m(1 To 3, 1 To 4) As Double possui quantos elementos?

- (A) 7
- (B) 12
- (C) 34
- (D) 43

Resposta: B

3 linhas × 4 colunas = 12 células.

Questão 5. Modularizar um programa significa:

- (A) colocar todo o código em um único formulário.
- (B) dividir o programa em partes (Sub/Function/Modules) reutilizáveis e organizadas.
- (C) usar apenas variáveis globais.
- (D) compilar em DOS.

Resposta: B

Modularização = coesão, reuso e manutenção facilitada.

Questão 6. Resultado da consulta SQL abaixo na tabela alunos(nome, nota) com linhas ('Ana',8), ('Beto',5), ('Carla',7):

Código para análise

PHP

```
SELECT nome FROM alunos
WHERE nota >= 7
ORDER BY nota DESC;
```

GABARITO

Ana e Carla, nesta ordem (8 antes de 7); Beto fica de fora pelo WHERE.

Critérios: Linhas corretas (1,5) + ordem DESC justificada (1,5).

Questão 7. O que o código exibe?

Código para análise

PHP

```
Dim m(1 To 2, 1 To 2) As Integer
Dim i As Integer, j As Integer
For i = 1 To 2
  For j = 1 To 2
    m(i, j) = i * j
  Next j
Next i
Debug.Print m(2, 2); m(1, 2); m(2, 1)
```

GABARITO

4 2 2 — preenchimento de matriz com laços aninhados (i*j).

Critérios: Valores corretos (2,0) + identificação de m(i,j)=i*j (1,0).

Questão 8. O trecho ADO abaixo entra em laço infinito ou falha. Corrija os 2 problemas.

Código para análise

PHP

```
Dim rs As ADODB.Recordset
Set rs = cn.Execute("SELECT nome FROM alunos")
Do While Not rs.EOF
List1.AddItem rs!nome
Loop
```

GABARITO — código modelo

```
Dim rs As ADODB.Recordset
Set rs = cn.Execute("SELECT nome FROM alunos")
Do While Not rs.EOF
List1.AddItem rs!nome
rs.MoveNext ' 1) avançar o registro
Loop
rs.Close ' 2) fechar o Recordset
```

Critérios: 1,5 pt MoveNext (sem ele, laço infinito); 1,5 pt Close (liberação de recursos).

Questão 9. Escreva os comandos (shell) de BACKUP e de RECUPERAÇÃO do banco escola em MySQL e explique em uma frase para que serve cada um.

GABARITO — código modelo

```
REM backup (cópia de segurança):
mysqldump -u root -p escola > escola_backup.sql
REM recuperação (restauração da cópia):
mysql -u root -p escola < escola_backup.sql
```

Critérios: mysqldump com redirecionamento (1,0) · mysql < para restaurar (1,0) · explicação correta de cada papel (1,0).

Questão 10. Escreva uma Sub RelatorioAlunos() que imprima com o objeto Printer: título centralizado com Tab, linha separadora com String(60,"-"), os nomes de um vetor v(1 To 5) numerados e, ao final, Printer.EndDoc.

GABARITO — código modelo

```
Public Sub RelatorioAlunos(v() As String)
Dim i As Integer
Printer.Print Tab(25); "RELATÓRIO DE ALUNOS"
Printer.Print String(60, "-")
For i = 1 To 5
Printer.Print i; " - "; v(i)
Next i
Printer.Print String(60, "-")
Printer.EndDoc
End Sub
```

Critérios: Cabeçalho com Tab (1,0) · separador String(60,"-") (0,5) · laço com dados (1,0) · EndDoc (0,5).

Rubrica do projeto integrador (10 pts — critério do professor)

Critério	Pontos	Descrição
Funcionalidade	3,0	Cadastro, consulta e relatório funcionando de ponta a ponta com o banco.

Critério	Pontos	Descrição
Estruturas e modularização	2,0	Uso coerente de vetor/matriz/registro e Sub/Function em Modules.
SQL e banco de dados	2,0	CREATE/INSERT/SELECT/UPDATE corretos; conexão tratada com falhas.
Relatório e saída	1,0	Relatório organizado (cabeçalho, linhas, totais) e/ou documento impresso.
Instalação e apresentação	2,0	Pacote de instalação gerado + apresentação clara com demonstração.

- As entregas 1–4 alimentam a nota de forma processual — acompanhe com vistos semanais.
- Na apresentação, exija demonstração com o banco populado e um relatório gerado ao vivo.
- O pacote de instalação (Entrega 4) pode ser demonstrado em máquina virtual se faltar tempo.

TERMOS ESSENCIAIS DA DISCIPLINA

Glossário

ADO	Biblioteca de acesso a dados do VB6 (Connection, Recordset) usada para falar com o SGBD.
Append (For Append)	Modo de abertura de arquivo que acrescenta linhas sem apagar o conteúdo.
AUTO_INCREMENT	Recurso que numera automaticamente a chave primária a cada INSERT.
Backup	Cópia de segurança da estrutura e dos dados do banco (ex.: mysqldump).
ByRef	Passagem de parâmetro por referência: a Sub altera a variável original do chamador.
ByVal	Passagem de parâmetro por valor: o parâmetro recebe uma cópia (original protegido).
Cliente/Servidor	Modelo em que a aplicação (cliente) requisita e o SGBD (servidor) processa e responde.
Close #n	Comando que fecha um arquivo aberto, descarregando o buffer de gravação.
cn.Execute	Método do ADODB.Connection que envia um comando SQL ao servidor.
Connection string	Texto com driver, servidor, banco e credenciais usado para abrir a conexão.
COUNT/SUM/AVG/MAX/MIN	Funções de agregação do SQL que resumem conjuntos de linhas.
CREATE TABLE	Comando DDL que define uma tabela (colunas, tipos, chave primária).
DDL	Família de comandos SQL de definição de estrutura (CREATE, ALTER, DROP).
DML	Família de comandos SQL de manipulação de dados (INSERT, UPDATE, DELETE, SELECT).
Do While / Do Until	Laços do VB6: repete enquanto verdadeiro / até virar verdadeiro.
Documento fiscal	Saída padronizada de venda (cupom/NF); no curso, modelo didático com Printer.
DSN/ODBC	Ponte de drivers entre a aplicação e o SGBD; DSN é o nome da conexão configurada.
EOF(n)	Função que indica fim de arquivo durante a leitura (condição de parada do laço).
Err.Description	Mensagem técnica do último erro em tempo de execução (tratamento com On Error).
For Output / For Input	Modos de abertura de arquivo: gravação (substitui) / somente leitura.
Format()	Função que formata números e datas para saída legível (ex.: "0.00", "dd/mm/yyyy").
FreeFile	Função que devolve um número de arquivo livre para Open.
Function	Rotina que retorna valor; o retorno é atribuído ao próprio nome.
GROUP BY	Cláusula SQL que agrupa linhas para agregações por grupo.
InStr	Função que devolve a posição de um trecho no texto (0 = não encontrou).
LIKE	Operador SQL de comparação com padrões e curingas (% e _).
Line Input #n	Leitura de uma linha inteira de arquivo texto.

Mid\$ / Left\$ / Right\$	Funções de recorte de texto por posição (contagem a partir de 1).
Module (.bas)	Arquivo de código com rotinas Public/Private reutilizáveis no projeto.
mysqldump	Utilitário que gera backup do banco em script .sql.
On Error GoTo	Estrutura de tratamento de erros em tempo de execução do VB6.
ORDER BY	Cláusula SQL de ordenação do resultado (ASC/DESC).
Package & Deployment Wizard	Ferramenta do VB6 que gera o pacote/disco de instalação.
PictureBox	Controle VB6 usado para recursos gráficos (Line, Circle, PSet).
Printer	Objeto VB6 que monta e envia saída à impressora (Print, Tab, EndDoc).
PRIMARY KEY	Coluna que identifica cada linha de forma única e não nula.
Recordset	Objeto ADO com o resultado de uma consulta; percorrido com EOF/MoveNext/rs!campo.
Registro (Type)	Tipo estruturado que agrupa campos de tipos diferentes (ficha).
Restore (recuperação)	Restauração de um backup: mysql banco < arquivo.sql.
Select Case	Estrutura de seleção para comparar um valor com casos fixos (menus).
Sentinela	Valor combinado que encerra uma leitura (ex.: -1).
SGBD	Sistema Gerenciador de Banco de Dados (MySQL, PostgreSQL, SQL Server, Access).
Split()	Função que quebra uma string em um vetor de partes por separador.
SQL	Linguagem padrão de definição e manipulação de dados em bancos relacionais.
String * n	Campo de texto de tamanho fixo em um Type (herança de arquivos de registro).
Sub	Rotina que executa ações sem retornar valor.
Trim\$ / UCase\$ / LCase\$	Funções de normalização de texto (pontas, caixa alta/baixa).
Vetor / Matriz	Arrays uni e bidimensionais: muitas posições sob um nome, acessadas por índice.
WHERE	Cláusula SQL que filtra linhas por condição.

OFICIAL DO PLANO DE ENSINO**Bibliografia****Básica**

- ALVES, William Pereira. Linguagem e lógica de programação. São Paulo: Erica, 2014.
- CARDOSO, Virginia M. Linguagem SQL. São Paulo: Saraiva, 2009.
- MILANI, Alessandra M. P. G. et al. Consultas em bancos de dados. Porto Alegre: SAGAH, 2021.

Complementar

- ALVES, William Pereira. Projetos de sistemas Web. São Paulo: Erica, 2019.
- GONÇALVES, Priscila de Fátima (Org.). Testes de software e gerência de configuração. Porto Alegre: SAGAH, 2019.
- JOSÉ, Marcel Fialho; REIS, Bruna de Souza. Projetos gráficos. São Paulo: Erica, 2015.
- LEDUR, Cleverson Lopes. Desenvolvimento de sistemas com C. Porto Alegre: SAGAH, 2018.
- MARTINS, Juliano V. S. et al. Raciocínio algorítmico. Porto Alegre: SAGAH, 2020.
- SIEBEL, Thomas M. Transformação digital. Rio de Janeiro: Alta Books, 2021.
- VELLOSO, Fernando de Castro. Informática. Rio de Janeiro: GEN LTC, 2017.

COMO LER ESTE DOCUMENTO

Itens marcados com (S) neste material são propostas pedagógicas de organização (datas de avaliações, entregas e distribuição das horas não presenciais) — não especificadas no Plano de Ensino original. As demais informações reproduzem o Plano de Ensino oficial da disciplina.

Bons estudos e bom código!

Linguagem de Programação II · Curso Técnico em Informática · Turma 2/2026 — material elaborado a partir do Plano de Ensino e do Cronograma oficiais (96 aulas · 24 semanas). Itens marcados com (S) são propostas pedagógicas.